

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Бизнес-проект «Программная реализация системы эвакуации маршрутов
для замкнутых пространств.» Программная реализация и тестирование
алгоритмов динамической маршрутизации

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

В. А. Старков

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

« ____ » _____ 20 ____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Старкова В. А., шифр 22-06-0111, группа ПО-216

1. Тема «Бизнес-проект «Программная реализация системы эвакуации маршрутов для замкнутых пространств.» Программная реализация и тестирование алгоритмов динамической маршрутизации» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) проанализировать предметную область навигации внутри зданий и определить требования к системе;

2) разработать архитектуру программного комплекса для построения маршрутов внутри здания с учетом этажности, переходов и ограничений на перемещение;

3) реализовать средства работы с навигационным графом, планами этажей, QR-навигацией и расписанием;

4) создать интерфейс пользователя и административные инструменты для редактирования карты, маршрутов и справочных данных;

5) предусмотреть поддержку режима чрезвычайной ситуации с построением эвакуационных маршрутов;

6) проверить корректность работы основных сценариев использования и подготовить материалы по результатам разработки.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: планы этажей в формате SVG, сведения о зданиях, этажах и навигационных узлах, навигационный граф, данные пользователей и их ролей, расписание занятий, параметры маршрутизации, а также состояние режима чрезвычайной ситуации.

2) Выходными данными для программной системы являются: построенный маршрут между выбранными точками, пошаговые инструкции перемещения, визуальное отображение маршрута на плане этажа, QR-данные для навигационных узлов, сведения о текущем расписании и эвакуационные маршруты при активации режима чрезвычайной ситуации.

4. Содержание работы (по разделам):

4.1. Введение.

4.2. Анализ предметной области: назначение систем построения эвакуационных маршрутов в реальном времени для замкнутых пространств, пользовательские роли, основные сценарии определения местоположения, особенности работы с картой здания, QR-навигацией, заблокированными участками и режимом чрезвычайной ситуации.

4.3. Техническое задание: основание для разработки, назначение системы построения эвакуационных маршрутов, требования к авторизации, построению маршрутов, отображению карты, QR-навигации, административным функциям, режиму чрезвычайной ситуации и оформлению программной документации.

4.4. Технический проект: общая характеристика системы построения эвакуационных маршрутов, обоснование выбора .NET MAUI, XAML и SkiaSharp, описание архитектуры MVVM, моделей навигационного графа, сервисного

слоя, пользовательского интерфейса, алгоритма построения маршрута и работы с локальными JSON-данными.

4.5. Рабочий проект: спецификация основных компонентов и классов приложения, описание экранов пользователя и администратора, проверка сценариев авторизации, выбора аудитории, построения маршрута, сканирования QR-кода, редактирования карты и активации режима чрезвычайной ситуации.

4.6. Заключение.

4.7. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цель и задачи разработки.

Лист 3. Диаграмма вариантов использования.

Лист 4. Концептуальная модель программы.

Лист 5. Построение маршрута между аудиториями.

Лист 6. Время построения маршрута в зависимости от размера графа.

Лист 7. Процесс построения маршрута по слоям приложения.

Лист 8. Заключение.

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

В. А. Старков

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 101 странице. Работа содержит 17 иллюстраций, 17 таблиц, 63 библиографических источников и 8 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: навигация, замкнутое пространство, эвакуационный маршрут, алгоритм Дейкстры, динамическая маршрутизация, навигационный граф, исключение узлов, межэтажный переход, чрезвычайная ситуация, кратчайший путь, QR-навигация, расписание, .NET MAUI, SkiaSharp, MVVM.

Объектом разработки является система построения эвакуационных маршрутов в реальном времени для замкнутых пространств, реализованная в виде кроссплатформенного приложения на платформе .NET MAUI.

Целью выпускной квалификационной работы является программная реализация и тестирование алгоритмов динамической маршрутизации, обеспечивающих построение оптимальных эвакуационных маршрутов с учётом динамически изменяющихся препятствий и режима чрезвычайной ситуации.

В ходе работы был проведён анализ алгоритмов поиска кратчайших путей на графах, разработаны структуры данных навигационного графа и механизмы перестроения маршрута в обход заблокированных узлов, реализованы алгоритм Дейкстры и алгоритм поиска ближайшего эвакуационного выхода.

Разработанная система позволяет формировать эвакуационные маршруты в реальном времени и служит основой для дальнейшего развития навигационных систем в замкнутых пространствах.

ABSTRACT

The volume of work is 101 page. The work contains 17 illustrations, 17 tables, 63 bibliographic sources and 8 sheets of graphic material. The number of applications is 2. The graphic material is presented in Annex A. Source code fragments are presented in Annex B.

List of keywords: navigation, enclosed space, evacuation route, Dijkstra's algorithm, dynamic routing, navigation graph, node exclusion, inter-floor transition, emergency situation, shortest path, QR navigation, schedule, .NET MAUI, SkiaSharp, MVVM.

The subject of this development is a system for constructing real-time evacuation routes for enclosed spaces, implemented as a cross-platform application on the .NET MAUI platform.

The goal of this final qualification project is the software implementation and testing of dynamic routing algorithms that ensure the construction of optimal evacuation routes, taking into account dynamically changing obstacles and emergency conditions.

During the project, an analysis of shortest-path search algorithms on graphs was conducted; data structures for the navigation graph and mechanisms for rerouting around blocked nodes were developed; and Dijkstra's algorithm and the nearest-exit search algorithm were implemented.

The developed system allows for the generation of evacuation routes in real time and serves as a foundation for the further development of navigation systems in enclosed spaces.

СОДЕРЖАНИЕ

РЕЗЮМЕ СТАРТАП-ПРОЕКТА	12
ВВЕДЕНИЕ	15
1 Анализ	18
1.1 Предметная область алгоритмов поиска путей	18
1.2 Обзор алгоритмов поиска кратчайших путей	19
1.3 Сравнительный анализ алгоритмов	21
1.4 Представление графа в памяти	22
1.5 Обоснование выбора алгоритма Дейкстры	23
1.6 Особенности применения алгоритма Дейкстры к эвакуационной маршрутизации	24
1.7 Анализ существующих реализаций	25
1.8 Выводы по разделу	26
2 Техническое задание	27
2.1 Основание для разработки	27
2.2 Назначение разработки	27
2.3 Требования к программной системе	28
2.3.1 Требования к данным программной системы	28
2.3.2 Требования к функциям программной системы	28
2.3.2.1 Построение маршрута между точками.	28
2.3.2.2 Динамическое исключение узлов.	29
2.3.2.3 Поиск эвакуационного выхода.	29
2.3.2.4 Обработка межэтажных переходов.	30
2.3.2.5 Восстановление пути.	30
2.3.3 Требования к структурам данных	30
2.3.3.1 Представление графа.	30
2.3.3.2 Узлы графа.	31
2.3.3.3 Рёбра графа.	31
2.3.4 Нефункциональные требования	31
2.4 Сценарии использования	32

2.4.1	Построение маршрута	33
2.4.2	Построение эвакуационного маршрута	34
2.4.3	Блокировка участка и перестроение маршрута	34
2.5	Требования к тестированию	35
2.5.1	Объём тестирования	35
2.5.2	Критерии приёмки	35
2.6	Требования к программной документации	35
3	Технический проект	37
3.1	Общая характеристика организации решения задачи	37
3.2	Обоснование выбора технологий реализации	39
3.2.1	Язык C# и платформа .NET.	39
3.2.2	Стандартные коллекции .NET.	39
3.2.3	Сериализация System.Text.Json.	39
3.3	Архитектура алгоритмического ядра	40
3.4	Структуры данных навигационного графа	41
3.4.1	Класс NavNode.	41
3.4.2	Класс NavEdge.	41
3.4.3	Класс NavGraph.	42
3.5	Проектирование хранения данных	43
3.6	Проектирование алгоритмов маршрутизации	45
3.6.1	Реализация алгоритма Дейкстры	45
3.6.2	Алгоритм поиска ближайшего эвакуационного выхода	46
3.6.3	Механизм формирования пошаговых инструкций	48
3.6.4	Множество исключённых узлов	48
3.7	Процесс построения маршрута	48
3.8	Проектирование пользовательского интерфейса	50
3.9	Тестирование алгоритмов	51
3.10	Сопоставление требований и компонентов	52
3.11	Выводы по о техническому проект	52
4	Рабочий проект	54
4.1	Общая характеристика рабочего проекта	54

4.2 Модули данных NavNode и NavEdge	55
4.3 Модуль NavGraph	56
4.4 Модуль NavGraphService	59
4.5 Модуль EmergencyService	60
4.6 Модуль MainViewModel	61
4.7 Модульное тестирование	63
4.7.1 Тестирование модуля построения маршрута	63
4.7.2 Тестирование модуля поиска эвакуационного выхода	65
4.7.3 Тестирование модуля вычисления весов рёбер	66
4.7.4 Тестирование модуля формирования множества исключений	67
4.8 Тестирование производительности	68
4.9 Системное тестирование	70
4.9.1 Построение маршрута между аудиториями	70
4.9.2 Эвакуационный маршрут в режиме ЧС	71
4.9.3 Перестроение маршрута при блокировке	72
4.9.4 Позиционирование пользователя по QR-коду	73
4.9.5 Определение исходной точки по расписанию в режиме ЧС	74
ЗАКЛЮЧЕНИЕ	76
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	77
ПРИЛОЖЕНИЕ А Представление графического материала	85
ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	94
На отдельных листах (CD-RW в прикрепленном конверте)	101
Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
Диаграмма вариантов использования (Графический материал / Диаграмма вариантов использования.png)	Лист 3
Концептуальная модель программы (Графический материал / Концептуальная модель программы.png)	Лист 4
Построение маршрута между аудиториями (Графический материал / Построение маршрута между аудиториями.png)	Лист 5

Время построения маршрута в зависимости от размера графа (Графический материал / Время построения маршрута в зависимости от размера графа.png) Лист 6

Процесс построения маршрута по слоям приложения (Графический материал / Процесс построения маршрута по слоям приложения.png) Лист 7

Заключение (Графический материал / Заключение.png) Лист 8

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

API – программный интерфейс приложения (Application Programming Interface).

C# – язык программирования, используемый для реализации логики приложения.

JSON (JavaScript Object Notation) – текстовый формат обмена и хранения структурированных данных.

MVVM (Model–View–ViewModel) – архитектурный шаблон, разделяющий модель данных, пользовательский интерфейс и модель представления.

QR-код – двумерный штрихкод, используемый для быстрого определения точки на карте.

UI (User Interface) – пользовательский интерфейс.

UX (User Experience) – пользовательский опыт.

XAML (Extensible Application Markup Language) – язык разметки пользовательского интерфейса в приложениях .NET MAUI.

Граф – структура данных, состоящая из узлов и рёбер, используемая для представления помещений, переходов и маршрутов.

ИС – информационная система.

ПО – программное обеспечение.

ТЗ – техническое задание.

ТП – технический проект.

ЧС – чрезвычайная ситуация.

SVG (Scalable Vector Graphics) – формат векторной графики, используемый для представления планов этажей.

UML (Unified Modelling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

РЕЗЮМЕ СТАРТАП-ПРОЕКТА

Название: «Бизнес-проект «Программная реализация системы эвакуации маршрутов для замкнутых пространств.» Программная реализация и тестирование алгоритмов динамической маршрутизации». Проект представляет собой кроссплатформенное приложение IndoorNav, предназначенное для навигации внутри зданий и автоматического построения эвакуационных маршрутов в чрезвычайных ситуациях. Система обеспечивает построение кратчайших маршрутов между помещениями, поиск ближайшего эвакуационного выхода, динамическое перестроение маршрута при блокировке участков пути, а также реализацию комплекса мер по обеспечению безопасности пользователей в режиме ЧС.

Ключевые возможности системы включают:

1. Построение кратчайших маршрутов между произвольными точками внутри здания с учётом межэтажных переходов и длин рёбер навигационного графа.
2. Автоматический поиск ближайшего эвакуационного выхода при активации режима чрезвычайной ситуации.
3. Перестроение маршрута в обход заблокированных участков без повторной загрузки графа.
4. Определение стартовой точки пользователя путём сканирования QR-кодов со стены здания.
5. Определение аудитории студента по расписанию и построение пути к выходу после её подтверждения.
6. Встроенный редактор графа, позволяющий администратору формировать карту здания без перекомпиляции приложения.

Разработанная система оптимизирует процесс эвакуации из замкнутых пространств, повышая безопасность пользователей за счёт оперативного построения оптимальных маршрутов к выходам с учётом заблокированных проходов.

Актуальность проекта обусловлена требованиями обеспечения безопасности людей в зданиях с большим количеством помещений и сложной планировкой. Бумажный план эвакуации статичен и не учитывает блокировку отдельных участков, что в условиях чрезвычайной ситуации может привести к выбору неоптимального или недоступного маршрута — цифровая система подобной ошибки не допускает.

Целью разработки является создание системы построения эвакуационных маршрутов в реальном времени, обеспечивающей:

- оперативное построение кратчайших маршрутов до ближайших выходов с учётом текущей обстановки;
- динамическое изменение маршрута при блокировке участков пути;
- работу на мобильных устройствах и настольных компьютерах;
- возможность масштабирования на здания с различной планировкой и этажности.

Отличительные особенности разработанной системы по сравнению с существующими решениями:

1. Алгоритм Дейкстры с возможностью динамического исключения узлов, позволяющий перестраивать маршрут без модификации исходного графа.
2. Кроссплатформенная реализация на базе .NET MAUI с поддержкой Android, iOS и Windows.
3. Полная автономность: данные хранятся локально, потеря сети не прерывает навигацию.
4. Редактор карты для администратора, позволяющий формировать навигационный граф поверх плана этажа.
5. Интеграция с расписанием занятий для автоматического определения местоположения студента при активации ЧС.

В системе реализованы алгоритмы маршрутизации (Дейкстра с исключением узлов, поиск ближайшего выхода), сервис управления режимом ЧС, аутентификация с разделением ролей (администратор, студент, гость), управление расписанием и позиционирование по QR-кодам.

Идея создания проекта возникла в 2025 году в связи с необходимостью обеспечения безопасной и оперативной эвакуации в зданиях университета. Разработка системы была начата в 2025 году и завершена в 2026 году в рамках выполнения выпускной квалификационной работы.

Бизнес-цели:

1. Внедрение системы навигации в зданиях университета для повышения безопасности студентов и сотрудников при чрезвычайных ситуациях.
2. Масштабирование системы для использования в зданиях различного назначения: образовательные учреждения, бизнес-центры, торговые комплексы.
3. Интеграция с системами пожарной безопасности и оповещения для автоматической активации режима ЧС.
4. Добавление push-уведомлений о чрезвычайных ситуациях и голосовые подсказки при навигации.
5. Ускорение процесса эвакуации за счёт автоматического определения местоположения пользователя.

ВВЕДЕНИЕ

Современные системы навигации в замкнутых пространствах требуют не только точного построения маршрутов, но и высокой производительности алгоритмов поиска пути, особенно в условиях динамически изменяющейся обстановки. В чрезвычайных ситуациях, когда определённые участки маршрута могут стать недоступными, система должна за миллисекунды перестроить оптимальный путь до ближайшего безопасного выхода.

Традиционные планы эвакуации решают только часть этой задачи. Они показывают общую схему движения, но остаются статичными: план не определяет местоположение конкретного пользователя, не учитывает заблокированный коридор и не перестраивает путь при изменении обстановки. При этом требования к эвакуационным путям и пожарной безопасности закреплены в нормативных документах, а значит программная система должна не подменять эти требования, а дополнять их прикладным механизмом построения маршрута.

В основе такой системы лежит представление здания в виде графа. Помещения, лестницы, лифты, выходы, QR-якоря и промежуточные точки маршрута рассматриваются как узлы, а допустимые переходы между ними – как рёбра. Это позволяет применять алгоритмы поиска кратчайшего пути, в том числе алгоритм Дейкстры, и учитывать дополнительные условия: смену этажа, недоступность отдельных участков, выбор ближайшего безопасного выхода. Теоретическая база подобных задач опирается на методы работы с графами и алгоритмы построения маршрутов.

Актуальность работы определяется необходимостью обеспечить надёжное и быстрое построение эвакуационных маршрутов в реальном времени. Алгоритмы маршрутизации должны обрабатывать графы с сотнями узлов и рёбер, учитывать межэтажные переходы, адаптироваться к изменениям в графе и гарантировать нахождение оптимального пути при наличии решения. Согласно критериям качества программного обеспечения по ГОСТ Р ИСО/МЭК 25010, в наибольшей степени проявляются характери-

ки функциональной пригодности (корректность маршрутизации), эффективности по производительности (время построения маршрута) и надёжности (устойчивость к блокировкам). Поэтому при реализации недостаточно только запрограммировать алгоритм: необходимо определить подходящие структуры данных, обеспечить обработку динамических ограничений и верифицировать корректность тестами.

Цель настоящей работы – реализовать и протестировать алгоритмы динамической маршрутизации для системы построения эвакуационных маршрутов, обеспечивающие построение оптимальных путей с учётом заблокированных участков и режима чрезвычайной ситуации.

Для достижения поставленной цели необходимо решить *следующие задачи*:

- изучить существующие алгоритмы поиска кратчайших путей на графах;
- спроектировать структуры данных для эффективного хранения и обработки навигационного графа;
- реализовать алгоритм Дейкстры с поддержкой динамического исключения узлов;
- реализовать алгоритм поиска ближайшего эвакуационного выхода с учётом текущего положения;
- построить механизм формирования множества исключаемых узлов для перестроения маршрута;
- провести модульное и системное тестирование алгоритмов на разных сценариях;
- оценить быстродействие и правильность работы алгоритмов.

Структура и объём работы – Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, приложений и графического материала на отдельных листах. Объём текста выпускной квалификационной работы составляет 101 страницу.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе на стадии анализа предметной области рассмотрены алгоритмы поиска кратчайших путей на графах, проведён их сравнительный анализ, рассмотрены способы представления графа в памяти и проанализированы существующие реализации систем indoor-навигации.

Во втором разделе на стадии технического задания приводятся требования к разрабатываемой программной системе.

В третьем разделе на стадии технического проектирования представлены проектные решения для программной системы.

В четвертом разделе приводится список функций, классов и их методов, использованных при разработке программной системы, производится тестирование разработанной программной системы

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода.

1 Анализ

1.1 Предметная область алгоритмов поиска путей

Задача построения оптимального маршрута в замкнутом пространстве сводится к поиску кратчайшего пути на взвешенном неориентированном графе. Вершины графа соответствуют помещениям, коридорам, лестничным площадкам и лифтам, а рёбра – проходам между ними с весами, пропорциональными расстоянию или времени прохождения. Такая модель является стандартной для систем indoor-навигации [40] и опирается на классическую теорию графов [47].

В отличие от наружной навигации, где маршруты строятся по дорожной сети с относительно простой топологией, навигация внутри здания характеризуется многоэтажностью, наличием переходов между этажами с различным расстоянием (лестницы, лифты), а также возможностью динамического изменения проходимости участков. При возникновении чрезвычайной ситуации определённые проходы могут стать заблокированными, что требует немедленного перестроения маршрута. Существующие подходы к навигации внутри зданий охватывают как методы определения положения пользователя – по Bluetooth, Wi-Fi, компьютерному зрению, QR-меткам [25, 60, 61], так и построение и сопровождение самого маршрута [37, 56, 59, 42]; настоящая работа сосредоточена на втором. Особую роль навигация играет при эвакуации: требования к путям и выходам закреплены в нормативных документах [6, 7], информационные технологии всё активнее применяются для повышения пожарной безопасности зданий [26], тогда как бумажные планы эвакуации статичны и требуют времени на изучение [44].

Предметная область алгоритмов маршрутизации в системах эвакуации включает следующие ключевые задачи:

- поиск кратчайшего пути между двумя заданными точками внутри здания;
- учёт весов рёбер, отражающих реальное расстояние или время прохождения;

- динамическое исключение заблокированных узлов без модификации исходного графа;
- поиск кратчайшего пути до ближайшего эвакуационного выхода из текущего положения;
- обработка межэтажных переходов с учетом веса ребра;
- обеспечение производительности, достаточной для работы в реальном времени.

1.2 Обзор алгоритмов поиска кратчайших путей

Для решения задачи поиска кратчайших путей на графах разработан ряд классических алгоритмов, каждый из которых имеет свои области применения, преимущества и ограничения. Ниже рассмотрены основные алгоритмы, применимые к задаче эвакуационной маршрутизации [20, 34, 32].

1. Алгоритм Дейкстры [27, 58] находит кратчайшие пути от одной вершины до всех остальных в графе с неотрицательными весами рёбер. Алгоритм работает следующим образом: на каждом шаге выбирается непосещённая вершина с минимальным текущим расстоянием, после чего выполняется релаксация всех исходящих из неё рёбер. Сложность алгоритма составляет $O((V + E) \log V)$ при использовании бинарной кучи или $O(E + V \log V)$ с кучей Фибоначчи, где V – число вершин, E – число рёбер. Алгоритм гарантирует оптимальность решения для графов с неотрицательными весами и является одним из наиболее изученных и надёжных алгоритмов поиска путей.

Достоинства алгоритма Дейкстры: гарантия оптимальности при неотрицательных весах; эффективная работа на разреженных графах; возможность остановки при достижении целевой вершины; простота модификации для учёта динамических ограничений; широкая распространённость и наличие проверенных реализаций.

Недостатки: не работает с отрицательными весами рёбер; требует полной инициализации расстояний для всех вершин; не учитывает направление к цели (в отличие от эвристических алгоритмов).

2. Алгоритм A^* является расширением алгоритма Дейкстры с использованием эвристической функции $h(v)$, оценивающей расстояние от вершины v до цели. Функция приоритета имеет вид $f(v) = g(v) + h(v)$, где $g(v)$ – текущее кратчайшее расстояние от старта, $h(v)$ – эвристическая оценка расстояния до цели. При допустимой эвристике ($h(v) \leq$ реальное расстояние) A^* гарантирует оптимальность и обычно посещает меньше вершин, чем Дейкстра.

Достоинства A^* : значительное сокращение области поиска при подходящей эвристике; оптимальность при допустимой эвристике; широкое применение в игровой индустрии и робототехнике.

Недостатки: требует разработки эвристической функции, что затруднительно для indoor-графов с произвольной топологией; эвристика на основе евклидова расстояния может быть неточной из-за стен и непроходимых участков; при динамическом исключении узлов эвристика может стать недопустимой; производительность сильно зависит от качества эвристики.

3. Алгоритм Флойда–Уоршелла находит кратчайшие пути между всеми парами вершин за $O(V^3)$. Алгоритм последовательно рассматривает каждую вершину как промежуточную и обновляет расстояния между всеми парами. Результат хранится в матрице расстояний, что позволяет получать путь между любыми двумя вершинами за $O(1)$.

Достоинства: полная матрица кратчайших путей; простота реализации; работа с отрицательными весами (без отрицательных циклов).

Недостатки: кубическая сложность делает алгоритм неприменимым для графов с сотнями вершин; при изменении одного ребра требуется полный пересчёт; высокое потребление памяти ($O(V^2)$); не подходит для динамической маршрутизации.

4. Поиск в ширину (BFS) обходит граф уровнями от стартовой вершины, гарантируя нахождение кратчайшего пути в количестве рёбер. BFS работает за $O(V + E)$ и является оптимальным для невзвешенных графов.

Достоинства: линейная сложность; простота реализации; нахождение кратчайшего пути по числу рёбер.

Недостатки: не учитывает веса рёбер, что делает его непригодным для графов с различной длиной проходов; не различает короткие коридоры и длинные лестницы; не применим для оптимизации по расстоянию или времени.

5. Алгоритм Беллмана–Форда находит кратчайшие пути от одной вершины до всех остальных, допуская отрицательные веса рёбер. Сложность составляет $O(V \cdot E)$. Алгоритм выполняет $V - 1$ итераций релаксации всех рёбер.

Достоинства: поддержка отрицательных весов; обнаружение отрицательных циклов.

Недостатки: значительно медленнее Дейкстры ($O(V \cdot E)$ vs $O((V + E) \log V)$); отрицательные веса не требуются в задаче навигации; избыточность для рассматриваемой задачи.

1.3 Сравнительный анализ алгоритмов

Сравнение рассмотренных алгоритмов по ключевым критериям приведены в таблицах 1.1 и 1.2.

Таблица 1.1 – Сравнение алгоритмов Дейкстры, A* и Флойда

Критерий	Дейкстра	A*	Флойд
Сложность	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V^3)$
Оптимальность	Да	Да*	Да
Отриц. веса	Нет	Нет	Да
Динамич. искл. узлов	Да	Затруднено	Нет
Требует эвристику	Нет	Да	Нет
Память	$O(V+E)$	$O(V+E)$	$O(V^2)$
Пересчёт при изменении	Полный	Полный	Полный

Таблица 1.2 – Сравнение алгоритмов BFS и Беллмана–Форда

Критерий	BFS	Беллман–Форд
Сложность	$O(V+E)$	$O(V \cdot E)$
Оптимальность	По рёбрам	Да
Отриц. веса	Нет	Да
Динамич. искл. узлов	Да	Да
Требует эвристику	Нет	Нет
Память	$O(V+E)$	$O(V+E)$
Пересчёт при изменении	Полный	Полный

1.4 Представление графа в памяти

Граф можно хранить двумя основными способами – матрицей смежности или списком смежности; выбор заметно влияет на расход памяти и скорость работы алгоритма.

Матрица смежности – это таблица $V \times V$, где на пересечении строки i и столбца j стоит длина ребра между вершинами i и j (или признак его отсутствия). Проверка наличия ребра выполняется за $O(1)$, но память расходуется как $O(V^2)$ независимо от числа рёбер, а перебор соседей одной вершины стоит $O(V)$.

Список смежности хранит для каждой вершины перечень её соседей с длинами. Память при этом – $O(V + E)$, а соседи перебираются ровно за их количество.

Граф здания разрежен: у каждого узла обычно два-четыре соседа, поэтому $E \approx 2-3V$. Сравнение двух представлений для такого случая приведено в таблице 1.3.

Таблица 1.3 – Сравнение способов представления графа

Характеристика	Матрица	Список
Память	$O(V^2)$	$O(V+E)$

Продолжение таблицы 1.3

Характеристика	Матрица	Список
Перебор соседей вершины	$O(V)$	$O(\deg v)$
Проверка наличия ребра	$O(1)$	$O(\deg v)$
Подходит для разреженных графов	Нет	Да

Алгоритм Дейкстры на каждом шаге перебирает соседей текущей вершины, а проверка отдельных рёбер ему почти не нужна. Поэтому для разреженного графа здания выбран список смежности: он экономит память (на 586 узлах матрица заняла бы около 340 тысяч ячеек против примерно 2,4 тысячи связей в списке) и ускоряет основной цикл алгоритма.

1.5 Обоснование выбора алгоритма Дейкстры

Для системы построения эвакуационных маршрутов выбран алгоритм Дейкстры по следующим причинам:

- эффективная работа на разреженных графах, характерных для планов зданий ($E \approx 2 - 3V$), в отличие от алгоритма Флойда–Уоршелла, требующего $O(V^3)$;
- возможность модификации для исключения заблокированных узлов без полного пересчёта структуры графа – достаточно передать множество исключений как параметр;
- простота реализации с использованием стандартных структур данных .NET (SortedSet, Dictionary, HashSet);
- достаточная производительность для графов с сотнями узлов (целевое время < 200 мс);
- отсутствие необходимости в эвристической функции, разработка которой затруднена для indoor-графов с произвольной топологией;
- возможность остановки при достижении целевой вершины без обработки всего графа.

Алгоритм A^* не выбран, поскольку для навигационного графа здания с произвольной конфигурацией стен и переходов трудно построить допустимую эвристику. Евклидово расстояние не учитывает непроходимые стены, а манхэттенское расстояние не соответствует реальной топологии коридоров. Кроме того, при динамическом исключении узлов эвристика может стать недопустимой, что нарушит гарантию оптимальности.

1.6 Особенности применения алгоритма Дейкстры к эвакуационной маршрутизации

Применение алгоритма Дейкстры к задаче эвакуации требует учёта следующих специфических требований, отличающих данную задачу от стандартного поиска кратчайшего пути.

Динамическое исключение узлов. В режиме чрезвычайной ситуации или при обходе препятствий определённые узлы графа (проходы, двери, лестницы) могут стать недоступными. Алгоритм должен исключать такие узлы из рассмотрения без модификации исходного графа. Это реализуется путём передачи множества исключённых идентификаторов как параметра алгоритма. При релаксации рёбер алгоритм проверяет, не находится ли соседняя вершина в множестве исключений, и пропускает её. Конечная вершина маршрута никогда не исключается, даже если она присутствует в множестве [57, 63].

Вес межэтажных переходов. Переход между этажами через лестницу или лифт имеет более большой вес ребра по сравнению с горизонтальным перемещением по коридору. Для межэтажных переходов назначается фиксированный вес, отражающий затраты времени на подъём/спуск. Данный подход позволяет алгоритму предпочитать горизонтальные маршруты, если они незначительно длиннее маршрута со сменой этажа [24].

Поиск до ближайшего выхода. Вместо поиска пути до заданной цели требуется найти кратчайший путь до любого из узлов, помеченных как эвакуационный выход. Это достигается запуском алгоритма Дейкстры от текущего положения пользователя с последующим сравнением расстояний до

всех доступных выходов. Выбирается выход с минимальной суммарной стоимостью пути. Приоритет отдаётся выходам в том же здании, что и текущее положение пользователя [54, 62].

Обработка отсутствия пути. Если после завершения алгоритма расстояние до целевой вершины остаётся бесконечным (вершина недостижима), алгоритм должен возвращать пустой список, а не генерировать исключение. Это важно для устойчивости системы: при полной блокировке пути приложение должно корректно информировать пользователя, а не завершаться аварийно [53].

1.7 Анализ существующих реализаций

На рынке существует ряд библиотек и фреймворков, предоставляющих реализацию алгоритмов поиска кратчайших путей. Готовые коммерческие платформы indoor-навигации [15, 16, 17, 18, 19] предоставляют навигацию внутри зданий как услугу, но они закрыты, требуют подписки и не дают доступа к реализации алгоритма маршрутизации. Поэтому ниже рассмотрены открытые алгоритмические библиотеки, наиболее известные из них.

1. NetworkX (Python) – библиотека для работы с графами, предоставляющая реализации алгоритмов Дейкстры, A^* , Флойда–Уоршелла и множества других. Широко используется в научных исследованиях и прототипировании. Однако библиотека написана на Python и не подходит для интеграции в .NET-приложение.

2. QuickGraph (C#) – библиотека графовых алгоритмов для .NET, предоставляющая реализации Дейкстры, A^* и других алгоритмов. Поддерживает ориентированные и неориентированные графы с произвольными типами вершин. Недостатком является отсутствие поддержки динамического исключения узлов из коробки, а также отсутствие поддержки .NET MAUI.

3. Boost.Graph (C++) – мощная библиотека графовых алгоритмов из набора Boost. Предоставляет высокопроизводительные реализации множества алгоритмов. Не подходит для .NET-приложения из-за языковых различий и сложности интеграции.

4. Собственная реализация – разработка алгоритма Дейкстры на C# с использованием стандартных коллекций .NET. Этот подход обеспечивает полный контроль над логикой алгоритма, возможность модификации для динамического исключения узлов, отсутствие внешних зависимостей и простоту отладки.

Для разрабатываемой системы выбрана собственная реализация, поскольку ни одна из существующих библиотек не предоставляет готовой поддержки динамического исключения узлов, необходимой для режима эвакуации. Кроме того, собственная реализация позволяет оптимизировать структуры данных под конкретную задачу и избежать лишних зависимостей.

1.8 Выводы по разделу

Проведённый анализ показывает, что алгоритм Дейкстры с использованием современных структур данных наилучшим образом соответствует требованиям задачи динамической эвакуационной маршрутизации. Его модификация для работы с динамически исключаемыми узлами и поиском ближайшего выхода позволяет создать надёжную основу для системы реального времени.

Ключевые результаты анализа:

- алгоритм Дейкстры обеспечивает оптимальность и достаточную производительность для графов зданий;
- алгоритм A* не подходит из-за сложности построения допустимой эвристики для indoor-графов;
- алгоритм Флойда–Уоршелла неприменим из-за кубической сложности;
- BFS не учитывает веса рёбер;
- собственная реализация предпочтительнее использования сторонних библиотек из-за необходимости поддержки динамического исключения узлов.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу по теме «Бизнес-проект «Программная реализация системы эвакуации маршрутов для замкнутых пространств.» Программная реализация и тестирование алгоритмов динамической маршрутизации». Разработка ведётся в соответствии с ГОСТ 19.102-77 [1] (стадии разработки) и ГОСТ 19.201-78 [3] (требования к техническому заданию).

Объектом разработки является алгоритмическое ядро системы построения эвакуационных маршрутов в реальном времени для замкнутых пространств, реализованное в виде модуля динамической маршрутизации приложения IndoorNav.

2.2 Назначение разработки

Функциональное назначение. Модуль динамической маршрутизации предназначен для:

- построения кратчайших маршрутов между точками внутри здания с учётом весов рёбер;
- поиска ближайшего эвакуационного выхода из текущего положения пользователя;
- динамического перестроения маршрута при блокировке участков пути;
- обработки межэтажных переходов;
- восстановления последовательности узлов маршрута для навигации пользователя.

Эксплуатационное назначение. Модуль предназначен для использования в составе кроссплатформенного приложения IndoorNav на мобильных устройствах (Android, iOS) и настольных компьютерах (Windows). В штатном режиме модуль обеспечивает построение маршрутов между помещени-

ями; в режиме ЧС – автоматическое построение эвакуационного маршрута до ближайшего безопасного выхода.

2.3 Требования к программной системе

2.3.1 Требования к данным программной системы

Входными данными для модуля маршрутизации являются:

- навигационный граф в формате JSON (узлы, рёбра, здания, этажи);
- идентификатор стартового узла (string);
- идентификатор конечного узла (string);
- множество исключённых узлов (ISet<string>, может быть пустым);
- текущее состояние режима ЧС (флаг активности, идентификатор здания).

Выходными данными модуля являются:

- упорядоченный список узлов маршрута от старта до цели (List<NavNode>);
- пустой список при отсутствии пути;
- суммарный вес маршрута (float);
- путь до ближайшего эвакуационного выхода при активации ЧС.

Концептуальная модель данных модуля включает сущности: навигационный узел (NavNode), навигационное ребро (NavEdge), навигационный граф (NavGraph). Навигационный узел имеет тип (аудитория, коридор, лестница, лифт, выход, запасной выход, QR-якорь) и координаты на плане этажа. Навигационное ребро связывает два узла и имеет вес, вычисляемый как евклидово расстояние или штраф для межэтажных переходов.

2.3.2 Требования к функциям программной системы

2.3.2.1 Построение маршрута между точками.

Модуль должен:

- строить кратчайший путь между двумя заданными узлами графа по алгоритму Дейкстры;

- учитывать веса рёбер при вычислении оптимального пути;
- корректно обрабатывать ситуацию отсутствия пути (несвязный граф, полная блокировка);
- обеспечивать время выполнения не более 200 мс для графа до 500 узлов и 1000 рёбер;
- возвращать упорядоченный список узлов маршрута (начало → конец);
- возвращать пустой список при совпадении стартового и конечного узла (путь из одного элемента);
- возвращать пустой список при несуществующих идентификаторах узлов.

2.3.2.2 Динамическое исключение узлов.

Алгоритм должен поддерживать:

- передачу множества исключаемых узлов как параметра без модификации исходного графа;
- игнорирование исключённых узлов при релаксации рёбер;
- невозможность исключения конечного узла маршрута (даже если он в множестве);
- корректную работу при последовательном добавлении исключений (повторный вызов);
- использование `HashSet<string>` для проверки принадлежности множеству за $O(1)$;
- возврат пустого результата при исключении всех возможных путей.

2.3.2.3 Поиск эвакуационного выхода.

Реализация должна обеспечивать:

- поиск кратчайшего пути до ближайшего узла с флагом `IsExit` или `IsEvacuationExit`;
- приоритет выходов в том же здании (`BuildingId`) при наличии нескольких корпусов;

- автоматическое построение маршрута при активации режима ЧС;
- возврат пустого результата при отсутствии доступных выходов;
- учёт исключённых узлов и заблокированных участков при поиске выхода;

- выбор выхода с минимальной суммой веса рёбер;
- fallback на выходы других зданий при отсутствии выходов в текущем.

2.3.2.4 Обработка межэтажных переходов.

Система должна:

- корректно обрабатывать рёбра, соединяющие узлы разных этажей;
- учитывать вес ребёр;
- избегать ненужных смен этажей при наличии прямого горизонтального пути;
- гарантировать достижимость любого этажа при отсутствии блокировок;
- различать типы переходов: лестница вверх, лестница вниз, лифт;
- формировать пошаговые инструкции при смене этажа.

2.3.2.5 Восстановление пути.

Алгоритм должен:

- сохранять предшественника для каждого достижимого узла;
- восстанавливать полный путь от старта до цели обратным обходом;
- возвращать путь в порядке от стартового узла к конечному;
- корректно обрабатывать случай, когда старт и цель совпадают.

2.3.3 Требования к структурам данных

2.3.3.1 Представление графа.

Граф должен храниться в формате, обеспечивающем:

- быстрый доступ к узлу по идентификатору через словарь (Dictionary<string, NavNode>);

- хранение списка смежности для каждого узла (Dictionary<string, List<(string, float)>>);

- хранение весов рёбер в формате float;
- поддержку неориентированных рёбер (двунаправленных);
- сериализацию/десериализацию в JSON для загрузки и сохранения.

2.3.3.2 Узлы графа.

Узел (NavNode) должен содержать:

- уникальный идентификатор (string);
- координаты на плане этажа (float X, Y);
- номер этажа (int);
- идентификатор здания (string);
- тип узла (обычный, переход, выход, эвакуационный выход, огнетушитель, QR-якорь);
- отображаемое название (string);
- необязательный идентификатор помещения;
- флаги: IsTransition, IsElevator, IsExit, IsEvacuationExit, IsQrAnchor;
- идентификатор группы перехода (TransitionGroupId) для связывания узлов на разных этажах.

2.3.3.3 Рёбра графа.

Ребро (NavEdge) должно содержать:

- идентификаторы соединяемых узлов (FromId, ToId);
- вес (евклидово расстояние или заданное значение);
- флаг межэтажного перехода (IsCrossFloor).

Вес ребра вычисляется автоматически: для узлов на одном этаже $w = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$, для узлов на разных этажах $w = 150$.

2.3.4 Нефункциональные требования

Требования к производительности.

- построение маршрута: не более 200 мс для графа до 500 узлов;

- поиск ближайшего выхода: не более 300 мс;
- потребление памяти: не более 50 МБ для графа здания;
- масштабируемость: поддержка графов до 1000 узлов и 2000 рёбер;
- построение списка смежности: не более 50 мс при загрузке графа.

Требования к надёжности.

- корректная работа при некорректных входных данных (несуществующие ID узлов);
- возврат пустого результата вместо исключения при отсутствии пути;
- устойчивость к циклическим исключениям узлов;
- обработка граничных случаев: граф из одного узла, старт = конец, петли;
- сохранение корректности при повторных вызовах с разными параметрами исключений;
- отсутствие утечек памяти при многократном построении маршрутов.

Требования к совместимости.

- совместимость с .NET 9 и MAUI workload;
- работа на Android 8.0+, iOS 15+ и Windows 10/11;
- использование только стандартных коллекций .NET (без нативных зависимостей).

2.4 Сценарии использования

С системой работают три типа пользователей (актёров):

- **Гость** – входит без регистрации; ему доступны построение маршрута, поиск ближайшего выхода, блокировка узла и навигация по QR-коду;
- **Студент** – авторизуется и вдобавок к возможностям гостя получает построение маршрута к аудитории из своего расписания;
- **Администратор** – размечает карту здания, управляет пользователями и включает режим ЧС по корпусам.

Распределение вариантов использования по ролям пользователей представлено на рисунке 2.1.

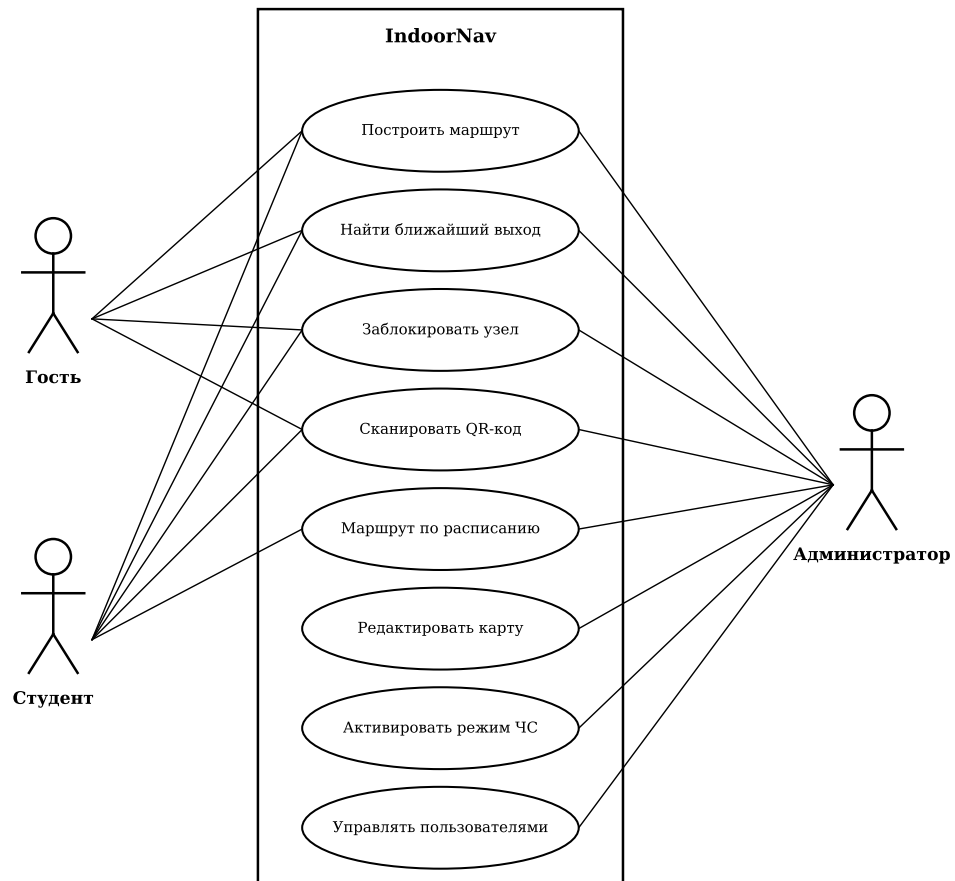


Рисунок 2.1 – Диаграмма вариантов использования

Ниже подробно разобраны три ключевых сценария: построение повседневного маршрута, эвакуация и перестроение маршрута при блокировке.

2.4.1 Построение маршрута

Сценарий навигации:

1. Пользователь выбирает начальную точку (вручную или через QR).
2. Пользователь выбирает конечную точку (поиск по названию или нажатием на карту).
3. ViewModel вызывает метод `NavGraph.FindPath(startId, endId, excludeIds)`.
4. Алгоритм Дейкстры строит кратчайший путь с учётом исключённых узлов.
5. Маршрут возвращается как упорядоченный список узлов.

6. ViewModel формирует пошаговые инструкции и отображает маршрут на карте.

2.4.2 Построение эвакуационного маршрута

Сценарий эвакуации:

1. Система получает от сервера флаг активации ЧС.
2. ViewModel вызывает метод `EmergencyService.FindNearestExitRoute(start, graph, blocked)`.
3. Сервис определяет все доступные эвакуационные выходы в текущем здании.
4. Для каждого выхода вычисляется кратчайший путь алгоритмом Дейкстры.
5. Маршрут возвращается как список узлов от текущего положения до выхода.
6. При отсутствии достижимых выходов возвращается пустой список.

2.4.3 Блокировка участка и перестроение маршрута

Сценарий обхода препятствия:

1. Пользователь нажимает на узел и выбирает «Заблокировать».
2. Идентификатор узла добавляется в множество заблокированных (`HashSet<string>`).
3. ViewModel формирует множество исключений: все выходы (кроме начального и конечного) и заблокированные узлы.
4. Вызывается метод `FindPath` с обновлённым множеством исключений.
5. Алгоритм строит маршрут в обход заблокированного узла.
6. Если путь отсутствует, то отображается сообщение «Маршрут не найден».

2.5 Требования к тестированию

2.5.1 Объём тестирования

Разработка должна включать:

- модульные тесты для алгоритма Дейкстры;
- модульные тесты для алгоритма поиска ближайшего выхода;
- тесты производительности (замеры времени выполнения на разных размерах графа);
- тесты с динамическим исключением узлов (последовательная блокировка пути);
- тесты на корректность весов рёбер (межэтажные переходы, евклидово расстояние).

2.5.2 Критерии приёмки

Модуль считается готовым при выполнении (согласно ГОСТ 19.301-79 [4]):

- успешного прохождения всех модульных и интеграционных тестов;
- соответствия требованиям производительности (замеры < 200 мс для 500 узлов);
- корректной работы на тестовых данных реального здания;
- документирования публичного API (классы NavGraph, NavNode, NavEdge, методы FindPath, FindNearestExitRoute);
- отсутствия утечек памяти при многократных вызовах.

2.6 Требования к программной документации

Программная документация должна соответствовать ГОСТ 19.105-78 [2] и включать:

- описание алгоритма Дейкстры и его модификаций для динамического исключения;
- описание структур данных навигационного графа;
- описание алгоритма поиска ближайшего эвакуационного выхода;

- описание механизма учёта межэтажных переходов;
- результаты тестирования производительности и корректности;
- рекомендации по оптимизации и дальнейшему развитию.

3 Технический проект

3.1 Общая характеристика организации решения задачи

Разрабатываемый модуль динамической маршрутизации является алгоритмическим ядром системы IndoorNav и обеспечивает построение оптимальных маршрутов внутри здания с учётом динамически изменяющихся препятствий. Модуль не зависит от пользовательского интерфейса и может использоваться как в мобильном приложении, так и в серверных компонентах.

Архитектура модуля следует принципу разделения ответственности: структуры данных графа выделены в отдельные классы (NavNode, NavEdge, NavGraph), алгоритмы поиска пути инкапсулированы в методах класса NavGraph, а специфичная логика эвакуации вынесена в сервис EmergencyService. Такое разделение упрощает тестирование: алгоритм Дейкстры можно проверять независимо от логики ЧС, а структуры данных – независимо от алгоритмов. Разделение ответственности между классами опирается на типовые приёмы объектно-ориентированного проектирования [23, 36] и принципы чистой архитектуры [39].

Система построена по архитектурному шаблону MVVM [10, 45] и разделена на четыре слоя: слой представления (View), слой моделей представления (ViewModel), сервисный слой (Service) и слой моделей данных (Model); ниже располагается хранилище данных в формате JSON. Схема архитектуры программной системы с указанием состава компонентов и связей между слоями приведена на рисунке 3.1. Компоненты, реализующие динамическую маршрутизацию (MainViewModel, NavGraphService, EmergencyService, NavGraph, NavNode, NavEdge, RouteStep), выделены на схеме цветом.

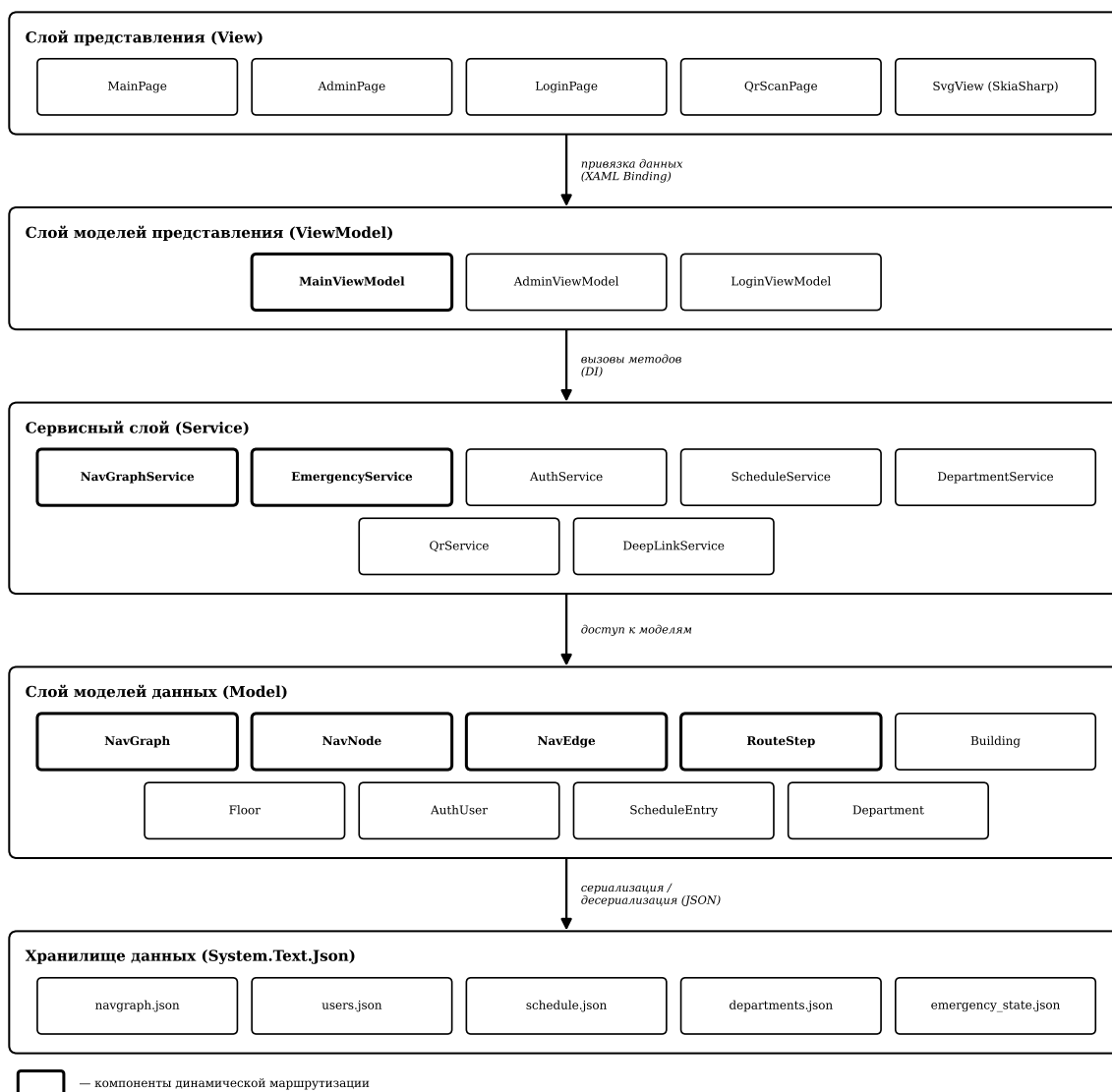


Рисунок 3.1 – Схема архитектуры программной системы IndoorNav

Каждый слой зависит только от нижележащего: представление взаимодействует с моделями представления через механизм привязки данных XAML [12], модели представления вызывают методы сервисов через внедрение зависимостей (DI), сервисы оперируют моделями данных, а сохранение и загрузка графа, пользователей, расписания и состояния ЧС выполняются сериализацией моделей в JSON средствами `System.Text.Json`. Такая слоистая организация изолирует алгоритмическое ядро от пользовательского интерфейса и платформенных особенностей [52, 38].

Главная особенность решения – поддержка динамического исключения узлов без модификации исходного графа. Это позволяет системе перестраивать маршруты в реальном времени при блокировке проходов, не перезагру-

жая данные и не создавая копии графа. Исключённые узлы передаются как параметр алгоритма и проверяются при обходе рёбер.

3.2 Обоснование выбора технологий реализации

3.2.1 Язык C# и платформа .NET.

Реализация выполнена на языке C# [48] в рамках платформы .NET MAUI [9], что обеспечивает единый код для Android, iOS и Windows. Кроссплатформенный подход выбран вместо нативной разработки под каждую ОС: он сохраняет единую кодовую базу, что подтверждается опытом перехода от Xamarin к .NET MAUI [28, 33] и сравнительными исследованиями кроссплатформенных мобильных технологий [31, 29, 51, 55]. C# предоставляет строгую типизацию для моделирования навигационного графа, обобщённые коллекции (Dictionary, SortedSet, HashSet) для эффективной реализации алгоритмов и LINQ для фильтрации узлов по зданию, этажу и типу.

3.2.2 Стандартные коллекции .NET.

Для реализации алгоритма Дейкстры используются стандартные коллекции .NET без привлечения сторонних библиотек: SortedSet<(float, string)> для очереди с приоритетом, Dictionary<string, float> для расстояний, Dictionary<string, string?> для предшественников, HashSet<string> для исключённых узлов. Это обеспечивает отсутствие внешних зависимостей и простоту отладки.

3.2.3 Сериализация System.Text.Json.

Загрузка и сохранение навигационного графа выполняется с использованием System.Text.Json [14] с настройкой WriteIndented = true. Формат JSON [8] выбран как компактный, читаемый и поддерживаемый на всех целевых платформах; среди форматов хранения текстовых данных он даёт хороший баланс между размером и удобством обработки [41].

3.3 Архитектура алгоритмического ядра

Модуль маршрутизации состоит из следующих компонентов:

- **NavNode** – класс узла графа с атрибутами (координаты, этаж, тип, флаги);
- **NavEdge** – класс ребра графа (связь между узлами с весом);
- **NavGraph** – класс графа, хранящий узлы и рёбра, предоставляющий методы поиска пути;
- **EmergencyService** – сервис поиска ближайшего эвакуационного выхода;
- **NavGraphService** – сервис загрузки, сохранения и предоставления графа.

Все компоненты регистрируются в контейнере внедрения зависимостей (MauiProgram.cs) как singleton [11], что обеспечивает единый экземпляр на всё время работы приложения и согласованность состояния. Внедрение зависимостей снижает связанность модулей и упрощает их раздельное тестирование [43, 49].

Схема взаимодействия слоёв MVVM с выделением алгоритмического модуля приведена на рисунке 3.2.

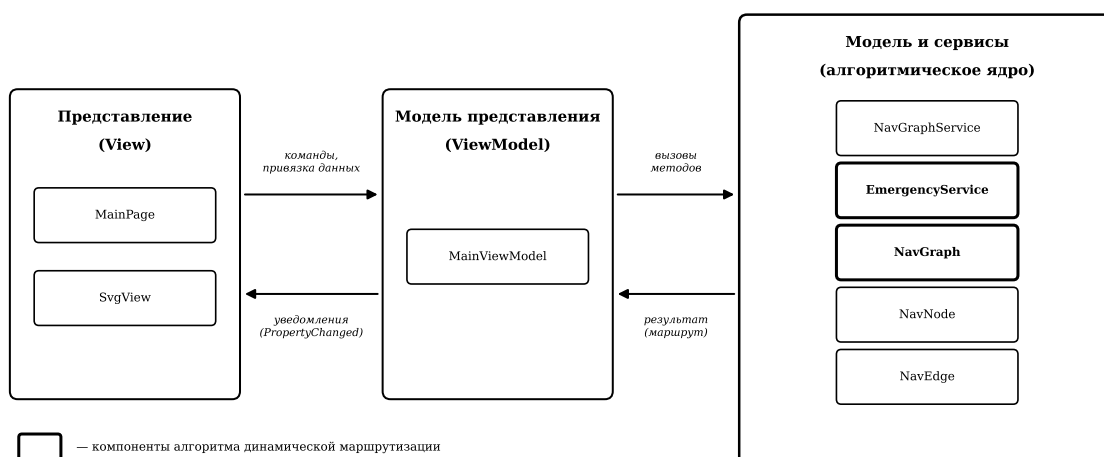


Рисунок 3.2 – MVVM-архитектура приложения с выделением алгоритмического модуля

3.4 Структуры данных навигационного графа

3.4.1 Класс NavNode.

Узел графа представляет точку на плане этажа (помещение, коридор, перекрёсток, лестничная площадка, выход) и содержит следующие свойства:

- Id – уникальный идентификатор узла (string);
- Name – отображаемое название для пользователя (string);
- X, Y – координаты на плане этажа в условных единицах (float);
- FloorNumber – номер этажа (int), используется для определения меж-этажных переходов;
- BuildingId – идентификатор здания/корпуса (string);
- IsTransition – флаг узла перехода между этажами;
- IsElevator – флаг лифтового узла;
- IsExit – флаг основного выхода;
- IsEvacuationExit – флаг эвакуационного выхода;
- IsFireExtinguisher – флаг огнетушителя (отображается при ЧС);
- IsQrAnchor – флаг QR-якоря для позиционирования;
- IsRoom – флаг узла-аудитории;
- IsWaypoint – флаг путевой точки;
- IsHidden – флаг скрытого узла (не отображается на карте);
- TransitionGroupId – идентификатор группы перехода для связывания узлов на разных этажах;
- RoomId – необязательный идентификатор помещения из внешней системы.

Узлы с флагами IsExit и IsEvacuationExit используются как целевые точки при построении эвакуационного маршрута.

3.4.2 Класс NavEdge.

Ребро графа представляет проход между двумя точками и содержит:

- FromId, ToId – идентификаторы соединяемых узлов;

- Weight – вес ребра, вычисляемый как расстояние между узлами с учётом типа прохода;
- IsCrossFloor – флаг межэтажного перехода.

Вес ребра вычисляется автоматически при создании: для узлов на одном этаже $w = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$, для узлов на разных этажах $w = 150$.

Фиксированный штраф 150 для межэтажных переходов отражает повышенные затраты времени на подъём/спуск по лестнице или лифту. Граф является неориентированным: при добавлении ребра создаются две направленные записи (от А к В и от В к А) с одинаковым весом.

3.4.3 Класс NavGraph.

Граф хранит коллекции узлов и рёбер и предоставляет алгоритмы поиска пути:

- Nodes – список всех узлов графа (List<NavNode>);
- Edges – список всех рёбер графа (List<NavEdge>);
- GetNode(id) – быстрый доступ к узлу по идентификатору через внутренний словарь;
- FindPath(startId, endId, excludeIds) – поиск кратчайшего пути методом Дейкстры;
- BuildAdjacency() – построение списка смежности для оптимизации поиска.

Метод BuildAdjacency() строит словарь связей Dictionary<string, List<(string neighbor, float weight)>>, который используется алгоритмом Дейкстры для быстрого доступа к соседям каждой вершины. Построение выполняется один раз при загрузке графа.

Пример навигационного графа здания приведён на рисунке 3.3. Узлы обозначены кружками с идентификаторами, рёбра – линиями с указанием веса. Узлы-выходы выделены двойной рамкой, узлы переходов между этажами – штриховой рамкой.

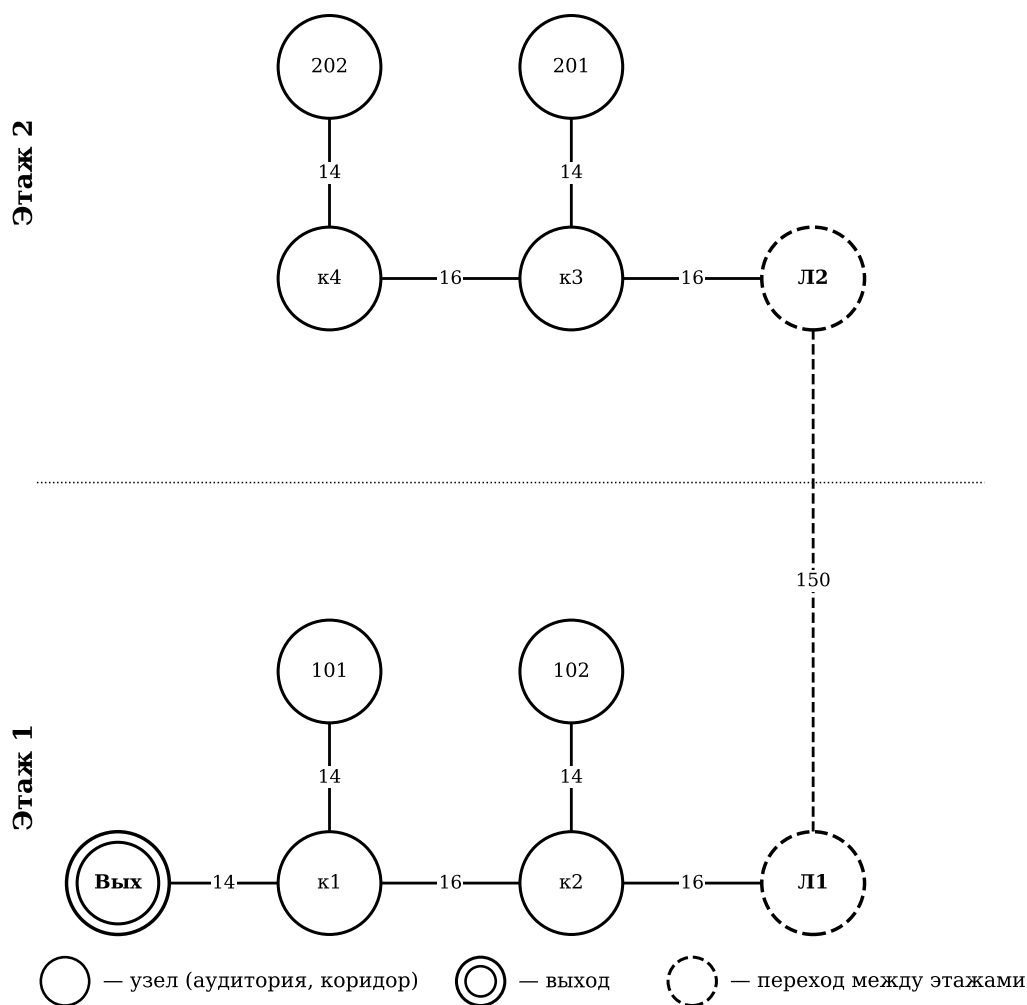


Рисунок 3.3 – Пример структуры навигационного графа здания

3.5 Проектирование хранения данных

Все данные приложения хранятся локально, в текстовых файлах формата JSON – так навигация не зависит от сети, а файлы при необходимости легко просмотреть и поправить вручную. Чтение и запись идут через `System.Text.Json`: объекты модели преобразуются в формат JSON и обратно без использования промежуточного слоя базы данных [22]. Данный подход оправдан для навигационного графа здания, поскольку граф полностью размещается в оперативной памяти, а его модификация происходит редко — преимущественно при разметке карты администратором.

Перечень файлов данных и их назначение приведены в таблице 3.1.

Таблица 3.1 – Файлы данных приложения

Файл	Содержимое
1	2
navgraph.json	Навигационный граф: массивы узлов и рёбер
users.json	Учётные записи с хешами паролей и ролями
departments.json	Кафедры и учебные группы
schedule.json	Расписание занятий по аудиториям и группам
emergency_state.json	Список зданий, где включён режим ЧС

Главный для маршрутизации файл – navgraph.json. На верхнем уровне это объект с двумя массивами: Nodes (узлы) и Edges (рёбра). Каждый узел хранит идентификатор, координаты, этаж, корпус и набор флагов типа; каждое ребро – идентификаторы концов, длина и признак межэтажного перехода. Структура напрямую повторяет классы NavNode и NavEdge, поэтому отдельная схема преобразования не нужна. Фрагмент файла с одним узлом и одним ребром выглядит так:

```

1 {
2   "Nodes": [
3     { "Id": "a-101", "Name": "101", "X": 320.5, "Y": 148.0,
4       "FloorNumber": 1, "BuildingId": "A",
5       "IsRoom": true, "IsExit": false, "IsEvacuationExit": false }
6   ],
7   "Edges": [
8     { "FromId": "a-101", "ToId": "a-kor1", "Weight": 42.3,
9       "IsCrossFloor": false }
10  ]
11 }

```

При запуске NavGraphService читает файл в объект NavGraph, после чего граф готов к работе. Сохранение выполняется обратной операцией с отступами (WriteIndented), чтобы файл оставался читаемым.

3.6 Проектирование алгоритмов маршрутизации

Алгоритмы маршрутизации включают четыре компонента: построение кратчайшего маршрута по алгоритму Дейкстры, поиск ближайшего эвакуационного выхода, формирование пошаговых инструкций и определение множества заблокированных узлов.

3.6.1 Реализация алгоритма Дейкстры

Реализация алгоритма Дейкстры в методе NavGraph.FindPath(startId, endId, excludeIds) использует следующие структуры данных:

- SortedSet<(float dist, string id)> – очередь с приоритетом для выбора вершины с минимальным расстоянием;
- Dictionary<string, float> – словарь текущих кратчайших расстояний от стартовой вершины;
- Dictionary<string, string?> – словарь предшественников для восстановления пути;
- ISet<string>? – множество исключаемых узлов (может быть null).

Динамическое исключение узлов. Множество исключаемых узлов передаётся как параметр excludeIds и проверяется при рассмотрении соседей. Это позволяет исключать узлы без модификации графа. Конечный узел

маршрута никогда не исключается, даже если его идентификатор присутствует в множестве.

Обработка отсутствия пути. Если после завершения алгоритма расстояние до целевой вершины остаётся равным `float.MaxValue` (вершина недостижима), метод возвращает пустой список. Это обеспечивает устойчивость системы: при полной блокировке пути приложение корректно информирует пользователя.

Оптимизация списка смежности. Перед запуском алгоритма строится словарь смежности `adj`, группирующий рёбра по исходящему узлу. Это позволяет получить всех соседей вершины за $O(1)$ вместо линейного поиска по списку рёбер.

Обработка устаревших записей очереди. При обновлении расстояния до вершины старая запись не удаляется из `SortedSet`, а проверяется при извлечении: если извлечённое расстояние превышает текущее, запись игнорируется. Это упрощает реализацию и избавляет от необходимости сложного обновления приоритетов.

3.6.2 Алгоритм поиска ближайшего эвакуационного выхода

Для режима чрезвычайной ситуации реализован алгоритм в методе `EmergencyService.FindNearestExitRoute(start, graph, excludeIds)`, который находит кратчайший путь до ближайшего эвакуационного выхода. Пример:

```

1 function FindNearestExitRoute(start, graph, excludeIds):
2
3     exits = graph.Nodes.Where(n =>
4         (n.IsExit or n.IsEvacuationExit)
5         and n.BuildingId == start.BuildingId
6         and excludeIds not contains n.Id)
7
8     if exits is empty:
9         exits = graph.Nodes.Where(n =>
10             (n.IsExit or n.IsEvacuationExit)
11             and excludeIds not contains n.Id)
12     if exits is empty:
13         return []
14
15     dist[start.Id] = 0
16     for each node in graph.Nodes:
17         if node.Id != start.Id:
18             dist[node.Id] = +infinity
19             prev[node.Id] = null
20     queue.Add((0, start.Id))
21     adj = BuildAdjacency(graph)
22
23     while queue is not empty:
24         (d, uid) = queue.ExtractMin()
25         if d > dist[uid]: continue
26         for each (nid, w) in adj[uid]:
27             if nid != start.Id and excludeIds contains nid:
28                 continue
29             alt = d + w
30             if alt < dist[nid]:
31                 queue.Remove((dist[nid], nid))
32                 dist[nid] = alt
33                 prev[nid] = uid
34                 queue.Add((alt, nid))
35
36     bestExit = exits.OrderBy(e => dist[e.Id]).First()
37     if bestExit == null or dist[bestExit.Id] == +infinity:
38         return []
39
40     path = []
41     current = bestExit.Id
42     while current != null:
43         path.Add(nodeMap[current])
44         current = prev[current]
45     path.Reverse()
46     return path

```

Интеграция с ViewModel. При активации режима ЧС MainViewModel вызывает ExecuteBuildEmergencyRoute(), который определяет текущее положение пользователя (StartNode), формирует множество заблокированных узлов (_blockedNodeIds), вызывает EmergencyService.FindNearestExitRoute(start, graph,

blocked), при наличии пути устанавливает конечный узел и вызывает `ExecuteBuildRoute()`, при отсутствии пути отображает сообщение «Маршрут до выхода не найден».

3.6.3 Механизм формирования пошаговых инструкций

После построения маршрута `MainViewModel.BuildRouteSteps(path)` формирует пошаговые текстовые инструкции для пользователя. Метод группирует узлы маршрута по этажам в сегменты: для каждого сегмента определяется направление движения и формируется текст инструкции. Для межэтажных переходов добавляется шаг «Перейдите на этаж N» с указанием типа перехода (лестница/лифт). Каждому шагу назначается иконка (прямо, поворот, лестница, лифт, выход), определяется фокусный узел и прямоугольник для подсветки на карте. После формирования обновляются свойства интерфейса: `RouteSteps`, `CurrentStep`, `HasRoute`.

3.6.4 Множество исключённых узлов

Метод `MainViewModel.BuildExcludeSet(start, end)` формирует множество узлов, исключаемых из маршрута. В множество включаются все узлы с флагом `IsExit` или `IsEvacuationExit`, кроме стартового и конечного, а также все узлы, заблокированные пользователем (`_blockedNodeIds`). Исключение выходов, не являющихся целевыми, предотвращает прохождение маршрута через другие выходы, что особенно важно в режиме эвакуации. Заблокированные узлы добавляются к множеству исключений, обеспечивая автоматическое перестроение маршрута в обход непроходимых участков.

3.7 Процесс построения маршрута

Описанные алгоритмы и структуры данных связываются в единый поток обработки запроса. Путь от действия пользователя до маршрута на экране проходит через все слои приложения и показан на рисунке 3.4.

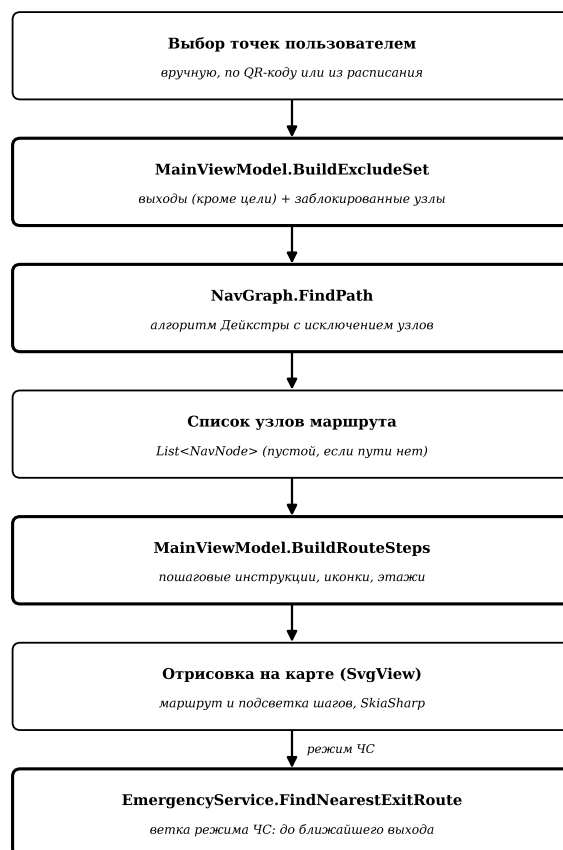


Рисунок 3.4 – Процесс построения маршрута по слоям приложения

Сначала пользователь задаёт начальную и конечную точки – вручную, сканированием QR-кода или выбором аудитории из расписания. Затем MainViewModel собирает множество исключаемых узлов (BuildExcludeSet) и передаёт его вместе с концами маршрута в NavGraph.FindPath. Алгоритм Дейкстры возвращает список узлов; если он пуст, пользователю выводится сообщение о недоступности пути. Непустой маршрут уходит в BuildRouteSteps, который превращает его в пошаговые инструкции, после чего SvgView отрисовывает путь и подсвечивает текущий шаг на плане этажа.

В режиме ЧС поток выполнения отличается: вместо обычного построения маршрута вызывается EmergencyService.FindNearestExitRoute, который самостоятельно определяет ближайший выход, после чего обработка продолжается по той же цепочке (список узлов, инструкции, отрисовка). Единый поток выполнения позволяет обоим сценариям, повседневному и эвакуационному, использовать один и тот же код отображения.

3.8 Проектирование пользовательского интерфейса

Интерфейс проектировался по принципам человеко-ориентированного проектирования и удобства пользователя [50, 46]: меньше действий до результата (маршрут строится не более чем в три касания), понятные подписи и адаптивная вёрстка под телефон и настольный компьютер. Разметка экранов описана на XAML, а данные связаны с моделями представления через привязку, поэтому одна и та же логика работает на всех платформах.

Приложение состоит из нескольких экранов:

1. **Экран входа.** Гость заходит одним нажатием, студент – по логину и паролю. Роль определяет доступные дальше возможности.

2. **Главный экран.** Занимает большую часть – интерактивный план этажа (контроль SvgView на SkiaSharp [13]) с панорамированием и масштабированием. Сверху – поля «Откуда» и «Куда» с поиском по списку узлов и автодополнением по названию и тегам, кнопка построения маршрута и переключатель здания и этажа. После построения снизу появляются карточки пошаговых инструкций с кнопками «Далее»/«Назад» и счётчиком шагов; при ЧС сверху выводится предупреждающий баннер.

3. **Экран QR-сканера.** Камера для считывания кода со стены (на Android и iOS); распознанный код задаёт начальную точку.

4. **Панель администратора.** Редактор карты (добавление и соединение узлов, выбор их типа), управление пользователями и расписанием, включение режима ЧС по корпусам.

Маршрут на плане выделяется отдельной линией с подсветкой текущего шага, а узлы-выходы и переходы между этажами обозначаются разными значками – чтобы пользователь различал их без подписей. Внешний вид главного экрана с построенным маршрутом приведён в разделе системного тестирования (рисунок 4.2).

3.9 Тестирование алгоритмов

Модульные тесты. Разработан набор модульных тестов, покрывающих следующие сценарии: построение простого маршрута (линейный граф из 3 узлов), выбор оптимального пути при наличии альтернатив (разветвлённый граф), корректная работа на графе с циклами, обработка несуществующих идентификаторов узлов (возврат пустого списка), построение маршрута при исключении одного узла, полная блокировка пути (исключение всех промежуточных узлов, возврат пустого результата), поиск ближайшего выхода из нескольких вариантов, работа с одноузловым графом (старт = конец, возврат списка из одного элемента), маршрут с межэтажным переходом (проверка штрафного веса 150), последовательное исключение нескольких узлов, исключение конечного узла (должен быть достижим независимо от множества), пустой граф (нет узлов, возврат пустого списка), несвязный граф (старт и цель в разных компонентах связности).

Тесты производительности. Проведены замеры времени выполнения на тестовых данных различного объёма: граф с 50 узлами и 80 рёбрами – среднее время 0,021 мс, граф с 200 узлами и 350 рёбрами – 0,098 мс, граф с 500 узлами и 900 рёбрами – 0,297 мс, поиск ближайшего выхода на графе 500 узлов – 0,630 мс. Все замеры укладываются в требование < 200 мс для построения маршрута и < 300 мс для поиска выхода с многократным запасом. Результаты показывают линейный рост времени выполнения с увеличением размера графа, что согласуется с теоретической сложностью $O((V + E) \log V)$ для разреженных графов. Подробные результаты и график зависимости приведены в рабочем проекте (таблица 4.12).

Интеграционные тесты. Проведено тестирование с реальными данными навигационного графа здания университета: загрузка графа из `navgraph.json` и корректная десериализация, построение маршрутов между реальными аудиториями, корректность весов рёбер (совпадение с визуальным расстоянием на плане), поиск эвакуационного маршрута из различных стартовых точек, перестроение маршрута при блокировке реальных проходов.

3.10 Сопоставление требований и компонентов

Сопоставление требований, сформулированных в техническом задании, и компонентов их реализации приведено в таблице 3.2.

Таблица 3.2 – Сопоставление требований и компонентов реализации

Требование	Компонент реализации
Поиск кратчайшего пути	NavGraph.FindPath() (алгоритм Дейкстры)
Динамическое исключение узлов	Параметр excludeIds в FindPath
Поиск ближайшего выхода	EmergencyService.FindNearestExitRoute()
Межэтажные переходы	Штрафной вес 150, флаг IsCrossFloor
Восстановление пути	Словарь prev + обратный обход
Структуры данных графа	NavNode, NavEdge, NavGraph
Сериализация	System.Text.Json, формат JSON
Пошаговые инструкции	MainViewModel.BuildRouteSteps()
Множество исключений	MainViewModel.BuildExcludeSet()
Производительность	SortedSet, Dictionary, список смежности

3.11 Выводы по о техническому проект

В техническом проекте описана реализация алгоритмического ядра системы IndoorNav: выбор технологий (C#, .NET, стандартные коллекции), структура модулей (NavNode, NavEdge, NavGraph, EmergencyService, NavGraphService), реализация алгоритма Дейкстры с поддержкой динамического исключения узлов, алгоритм поиска ближайшего эвакуационного выхода, механизм формирования пошаговых инструкций, формирование множества исключённых узлов, результаты модульного тестирования и замеров производительности, сериализация графа в JSON.

Разработанное алгоритмическое ядро обеспечивает эффективное построение эвакуационных маршрутов с учётом динамических препятствий. Использование алгоритма Дейкстры с современными структурами данных гарантирует производительность, соответствующую требованиям реального времени.

4 Рабочий проект

4.1 Общая характеристика рабочего проекта

Если в техническом проекте приведены проектные решения и псевдокод алгоритмов динамической маршрутизации, то в рабочем проекте представлена их программная реализация на языке C# в составе приложения IndoorNav и результаты комплексного тестирования. Реализация выполнена в соответствии с проектными решениями технического проекта (таблица 3.2) и сохраняет принцип разделения ответственности: структуры данных, алгоритмы поиска пути и логика эвакуации реализованы как независимые модули, что обеспечивает возможность их раздельного тестирования.

В процессе разработки алгоритмического ядра динамической маршрутизации выделены следующие программные модули: NavNode, NavEdge, NavGraph, NavGraphService, EmergencyService и MainViewModel. Их состав и распределение по слоям архитектуры приведены в таблице 4.1.

Таблица 4.1 – Состав программных модулей алгоритмического ядра

Модуль (файл)	Слой	Назначение
1	2	3
NavNode.cs	Model	Класс узла навигационного графа
NavEdge.cs	Model	Класс ребра графа с весом
NavGraph.cs	Model	Граф и алгоритм Дейкстры (FindPath)
NavGraphService.cs	Service	Загрузка, сохранение и предоставление графа
EmergencyService.cs	Service	Поиск ближайшего эвакуационного выхода
MainViewModel.cs	ViewModel	Построение маршрута, множество исключений, пошаговые инструкции

Далее каждый модуль рассмотрен отдельно: приведены его назначение и таблица реализованных полей или методов с описанием параметров.

4.2 Модули данных NavNode и NavEdge

NavNode – класс узла навигационного графа, представляющий точку на плане этажа (помещение, коридор, перекрёсток, лестничную площадку или выход). NavEdge – класс ребра графа, представляющий проход между двумя узлами с весом. Это модули данных без вычислительной логики; их ключевые поля приведены в таблицах 4.2 и 4.3.

Таблица 4.2 – Ключевые поля класса NavNode

Поле	Тип	Описание
1	2	3
Id	string	Уникальный идентификатор узла
X, Y	float	Координаты узла на плане этажа
FloorNumber	int	Номер этажа
BuildingId	string	Идентификатор здания (корпуса)
IsTransition	bool	Узел межэтажного перехода (лестница, лифт)
IsExit	bool	Основной выход из здания
IsEvacuationExit	bool	Эвакуационный выход
TransitionGroupId	string	Группа перехода для связывания узлов разных этажей

Таблица 4.3 – Поля класса NavEdge

Поле	Тип	Описание
FromId, ToId	string	Идентификаторы соединяемых узлов
Weight	float	Вес ребра (евклидово расстояние или штраф 150)
IsCrossFloor	bool	Признак межэтажного ребра

4.3 Модуль NavGraph

NavGraph – центральный модуль алгоритмического ядра: хранит коллекции узлов и рёбер и реализует поиск кратчайшего пути по алгоритму Дейкстры с поддержкой динамического исключения узлов. Описание реализованных методов приведено в таблице 4.4.

Таблица 4.4 – Методы модуля NavGraph

Метод	Аргументы	Описание
1	2	3
GetNode	id	Возвращает узел по идентификатору или null
FindPath	startId, endId, excludeIds	Поиск кратчайшего пути по алгоритму Дейкстры с динамическим исключением узлов
AddEdge	fromId, toId, crossFloor	Добавление ребра с автоматическим вычислением веса в обоих направлениях
RemoveEdge	fromId, toId	Удаление ребра между двумя узлами
EdgeExists	fromId, toId	Проверка наличия ребра между узлами
GetNodesForFloor	buildingId, floor	Узлы заданного этажа корпуса
GetEdgesForFloor	buildingId, floor	Рёбра заданного этажа корпуса

Реализация поиска кратчайшего пути по алгоритму Дейкстры в методе FindPath принимает на вход идентификаторы стартового и конечного узлов, а также множество исключаемых узлов excludeIds. Пример:

```

1 public List<NavNode> FindPath(string startId, string endId, ISet<string>?
   excludeIds)
2 {
3     if (startId == endId)
4         return GetNode(startId) is { } n ? [n] : [];
5
6     var dist = new Dictionary<string, float>();
7     var prev = new Dictionary<string, string?>();
8     var queue = new SortedSet<(float dist, string id)>(<
9         Comparer<(float, string)>.Create((a, b) =>
10         a.Item1 != b.Item1 ? a.Item1.CompareTo(b.Item1)
11             : string.Compare(a.Item2, b.Item2,
12                 StringComparison.Ordinal));
13
14     foreach (var node in Nodes)
15     {
16         dist[node.Id] = float.MaxValue;
17         prev[node.Id] = null;
18     }
19     dist[startId] = 0;
20     queue.Add((0, startId));
21
22     var adj = Nodes.ToDictionary(n => n.Id, _ => new List<(string neighbor,
23         float weight)>());
24     foreach (var edge in Edges)
25         if (adj.ContainsKey(edge.FromId))
26             adj[edge.FromId].Add((edge.ToId, edge.Weight));
27
28     while (queue.Count > 0)
29     {
30         var (d, u) = queue.Min;
31         queue.Remove(queue.Min);
32         if (u == endId) break;
33         if (d > dist[u]) continue;
34
35         foreach (var (v, w) in adj[u])
36         {
37             if (excludeIds != null && excludeIds.Contains(v) && v != endId)
38                 continue;
39             float alt = dist[u] + w;
40             if (alt < dist[v])
41             {
42                 queue.Remove((dist[v], v));
43                 dist[v] = alt;
44                 prev[v] = u;
45                 queue.Add((alt, v));
46             }
47         }
48     }
49
50     if (dist[endId] == float.MaxValue) return [];
51 }

```

Очередь с приоритетом реализована на основе SortedSet с компаратором, сравнивающим элементы сначала по расстоянию, а при равенстве – по идентификатору узла. Это гарантирует строгий порядок элементов и детерминированность результата. Список смежности adj строится один раз перед основным циклом, что обеспечивает доступ к соседям вершины за константное время. Устаревшие записи очереди не удаляются при обновлении расстояния, а отбрасываются при извлечении проверкой $d > \text{dist}[u]$.

4.4 Модуль NavGraphService

NavGraphService – сервисный модуль, отвечающий за загрузку, сохранение и предоставление навигационного графа, а также за операции добавления и удаления узлов и рёбер. Сериализация графа выполняется средствами System.Text.Json. Описание методов приведено в таблице 4.5.

Таблица 4.5 – Методы модуля NavGraphService

Метод	Аргументы	Описание
1	2	3
LoadAsync	—	Загрузка графа из файла navgraph.json
SaveAsync	—	Сохранение графа в файл navgraph.json
ResetAsync	—	Сброс графа и удаление файла данных
AddNode	node	Добавление узла в граф
RemoveNode	id	Удаление узла вместе с инцидентными рёбрами
AddEdge	fromId, toId, crossFloor	Добавление ребра в граф

Продолжение таблицы 4.5

Метод	Аргументы	Описание
RemoveEdge	fromId, toId	Удаление ребра из графа

4.5 Модуль EmergencyService

EmergencyService – сервисный модуль режима чрезвычайной ситуации: управляет состоянием ЧС по зданиям и реализует поиск кратчайшего пути до ближайшего эвакуационного выхода. Описание методов приведено в таблице 4.6.

Таблица 4.6 – Методы модуля EmergencyService

Метод	Аргументы	Описание
1	2	3
FindNearestExitRoute	start, graph, excludeIds	Поиск пути до ближайшего эвакуационного выхода
Activate	buildingId	Активация режима ЧС для здания
Deactivate	buildingId	Отключение режима ЧС для здания
ActivateAll	buildingIds	Активация ЧС для всех зданий
IsActiveForBuilding	buildingId	Проверка активности ЧС для здания
LoadAsync	—	Восстановление состояния ЧС из файла

Реализация поиска ближайшего эвакуационного выхода выполнена в методе FindNearestExitRoute: он отбирает множество узлов-выходов, запускает однократный обход Дейкстры от стартовой точки и выбирает выход с минимальным расстоянием. Пример:

```
1 var exits = graph.Nodes
2   .Where(n => (n.IsExit || n.IsEvacuationExit)
3               && n.BuildingId == start.BuildingId
4               && excludeIds?.Contains(n.Id) != true)
5   .ToList();
6
7 var bestExit = exits.OrderBy(e => dist.GetValueOrDefault(e.Id, double.
8   MaxValue))
9   .FirstOrDefault();
10
11 if (bestExit == null || dist[bestExit.Id] == double.MaxValue) return new();
12
13 var path = new List<NavNode>();
14 string? cur = bestExit.Id;
15 while (cur != null)
16 {
17     if (nodeMap.TryGetValue(cur, out var node)) path.Add(node);
18     prev.TryGetValue(cur, out cur);
19 }
20 path.Reverse();
21 return path;
```

Приоритизация выходов текущего здания исключает построение маршрута в соседний корпус при наличии ближайшего выхода. Резервный поиск по всем корпусам гарантирует нахождение выхода в нестандартной конфигурации данных. Заблокированные узлы исключаются на этапе отбора выходов и при релаксации рёбер.

4.6 Модуль MainViewModel

MainViewModel — модуль слоя представления, связывающий пользовательский интерфейс с алгоритмами маршрутизации: формирует множество исключаемых узлов, вызывает построение маршрута и формирует пошаговые инструкции. Описание ключевых методов приведено в таблице 4.7.

Таблица 4.7 – Ключевые методы модуля MainViewModel

Метод	Аргументы	Описание
1	2	3
ExecuteBuildRoute	—	Построение маршрута между выбранными точками
ExecuteBuildEmergencyRoute	—	Построение эвакуационного маршрута в режиме ЧС
BuildExcludeSet	startId, endId	Формирование множества исключаемых узлов
BuildRouteSteps	path	Формирование пошаговых инструкций по маршруту

Формирование множества исключаемых узлов выполняется методом BuildExcludeSet, объединяющим выходы, не являющиеся целевыми, и узлы, заблокированные пользователем.

```

1 private HashSet<string> BuildExcludeSet(string startId, string endId)
2 {
3     var exclude = new HashSet<string>();
4     foreach (var n in _graph.Nodes)
5         if ((n.IsExit || n.IsEvacuationExit) && n.Id != startId && n.Id !=
6             endId)
7             exclude.Add(n.Id);
8     foreach (var id in _blockedNodeIds)
9         exclude.Add(id);
10    return exclude;
11 }

```

Объединение двух источников исключений в одном множестве позволяет единообразно передавать ограничения в метод FindPath и обеспечивает автоматическое перестроение маршрута в обход заблокированных участков.

4.7 Модульное тестирование

Для проверки корректности разработаны модульные тесты для четырёх модулей, образующих алгоритмическое ядро маршрутизации: построения маршрута (`NavGraph.FindPath`), поиска ближайшего эвакуационного выхода (`EmergencyService.FindNearestExitRoute`), вычисления весов рёбер (`NavGraph.AddEdge`) и формирования множества исключений (`MainViewModel.BuildExcludeSet`). Для каждого модуля составлена отдельная таблица тестовых наборов, в которой указаны сценарий, входные данные и эталонный выходной результат.

4.7.1 Тестирование модуля построения маршрута

Тестовые наборы для модуля построения маршрута охватывают типовые и граничные сценарии и приведены в таблице 4.8. Пример реализации теста построения простого маршрута на линейном графе приведён ниже.

```
1 [Test]
2 public void FindPath_LinearGraph_ReturnsFullPath()
3 {
4     var graph = new NavGraph();
5     graph.Nodes.Add(new NavNode { Id = "A", X = 0, Y = 0 });
6     graph.Nodes.Add(new NavNode { Id = "B", X = 10, Y = 0 });
7     graph.Nodes.Add(new NavNode { Id = "C", X = 20, Y = 0 });
8     graph.AddEdge("A", "B");
9     graph.AddEdge("B", "C");
10
11     var path = graph.FindPath("A", "C", null);
12
13     Assert.AreEqual(3, path.Count);
14     Assert.AreEqual("A", path.First().Id);
15     Assert.AreEqual("C", path.Last().Id);
16 }
```

Таблица 4.8 – Тестовые наборы для модуля построения маршрута (NavGraph.FindPath)

№	Сценарий	Входные данные	Эталонные выходные данные
1	2	3	4
1	Простой маршрут	Линейный граф из узлов А, В, С; запрос $A \rightarrow C$	Путь из трёх узлов [А, В, С]
2	Выбор оптимального пути	Граф с двумя ветвями разной длины	Кратчайшая из ветвей
3	Граф с циклом	Граф, содержащий цикл	Кратчайший путь без заикливания
4	Несуществующий узел	Отсутствующий стартовый или конечный идентификатор	Пустой список
5	Исключение узла	Множество исключений с одним промежуточным узлом	Маршрут в обход исключённого узла
6	Полная блокировка пути	В множество исключений добавлены все промежуточные узлы	Пустой список
7	Старт совпадает с целью	Стартовый и конечный узел совпадают	Список из одного узла
8	Межэтажный переход	Граф с межэтажным ребром (вес 150)	Путь с учётом штрафного веса 150
9	Последовательное исключение	Повторные вызовы с растущим множеством исключений	Корректно перестроенный маршрут

Продолжение таблицы 4.8

№	Сценарий	Входные данные	Эталонные выходные данные
10	Исключение конечно-го узла	Конечный узел добавлен в множество исключений	Конечный узел достигим, путь построен
11	Пустой граф	Граф без узлов	Пустой список
12	Несвязный граф	Старт и цель в разных компонентах связности	Пустой список

4.7.2 Тестирование модуля поиска эвакуационного выхода

Тестовые наборы для модуля поиска ближайшего эвакуационного выхода учитывают приоритет выходов текущего здания, резервный поиск по всем корпусам и исключение заблокированных выходов; они приведены в таблице 4.9.

Таблица 4.9 – Тестовые наборы для модуля поиска эвакуационного выхода (EmergencyService.FindNearestExitRoute)

№	Сценарий	Входные данные	Эталонные выходные данные
1	2	3	4
1	Несколько выходов	Граф с двумя и более выходами; стартовый узел	Путь до ближайшего выхода
2	Единственный выход	Граф с одним узлом-выходом	Путь до этого выхода
3	Нет выходов	Граф без узлов-выходов	Пустой список

Продолжение таблицы 4.9

№	Сценарий	Входные данные	Эталонные выходные данные
4	Выход в другом корпусе	В текущем здании выходов нет	Путь до выхода другого корпуса (резервный поиск)
5	Заблокированный выход	Ближайший выход добавлен в множество исключений	Путь до следующего по близости выхода

4.7.3 Тестирование модуля вычисления весов рёбер

Модуль вычисления весов рёбер (NavGraph.AddEdge) формирует вес ребра как евклидово расстояние между узлами одного этажа либо как фиксированный штраф 150 для межэтажных переходов и добавляет ребро в обоих направлениях. От корректности весов напрямую зависит оптимальность маршрутов, поэтому модуль тестируется отдельно. Тестовые наборы приведены в таблице 4.10.

Таблица 4.10 – Тестовые наборы для модуля вычисления весов рёбер (NavGraph.AddEdge)

№	Сценарий	Входные данные	Эталонные выходные данные
1	2	3	4
1	Вес горизонтального ребра	Узлы с координатами (0, 0) и (3, 4) на одном этаже	Вес ребра = 5,0 (евклидово расстояние)
2	Вес межэтажного ребра	Добавление ребра с признаком межэтажного перехода	Вес ребра = 150

Продолжение таблицы 4.10

№	Сценарий	Входные данные	Эталонные выходные данные
3	Двунаправленность ребра	Добавление ребра между узлами А и В	Создаются рёбра $A \rightarrow B$ и $B \rightarrow A$ с равным весом
4	Защита от дублирования	Повторное добавление уже существующего ребра	Повторное ребро не создаётся
5	Несуществующий узел	Добавление ребра с отсутствующим идентификатором узла	Ребро не создаётся

4.7.4 Тестирование модуля формирования множества исключений

Модуль формирования множества исключений (`MainViewModel.BuildExcludeSet`) объединяет узлы-выходы, не являющиеся целевыми, и заблокированные пользователем узлы; именно он обеспечивает динамическое перестроение маршрута в обход недоступных участков. Тестовые наборы приведены в таблице 4.11.

Таблица 4.11 – Тестовые наборы для модуля формирования множества исключений (MainViewModel.BuildExcludeSet)

№	Сценарий	Входные данные	Эталонные выходные данные
1	2	3	4
1	Исключение прочих выходов	Граф с несколькими выходами; старт и цель не являются выходами	В множество включены все узлы-выходы
2	Стартовый выход не исключается	Стартовый узел помечен как выход	Стартовый узел отсутствует в множестве
3	Целевой выход не исключается	Конечный узел помечен как выход	Конечный узел отсутствует в множестве
4	Учёт заблокированных узлов	Список заблокированных узлов содержит узел X	Узел X включён в множество
5	Объединение источников	Заданы и узлы-выходы, и заблокированные узлы	Множество = выходы (кроме старта и цели) и заблокированные узлы

4.8 Тестирование производительности

Для подтверждения соответствия требованиям реального времени проведены замеры времени построения маршрута на графах различного объёма. Замеры выполнены на ПК с процессором AMD Ryzen 7 5700X в среде выполнения .NET 9 (конфигурация Release); для каждого размера графа результат усреднён по 20000 построений маршрута между случайными парами узлов на 5 различных связных разреженных графах. Результаты замеров приведе-

ны в таблице 4.12, графическая зависимость времени выполнения от размера графа – на рисунке 4.1.

Таблица 4.12 – Результаты тестирования производительности

Узлы	Рёбра	Среднее время, мс	Требование
50	80	0,021	< 200 мс
100	160	0,035	< 200 мс
150	250	0,065	< 200 мс
200	350	0,098	< 200 мс
300	520	0,147	< 200 мс
400	700	0,218	< 200 мс
500	900	0,297	< 200 мс
500 (поиск выхода)	900	0,630	< 300 мс

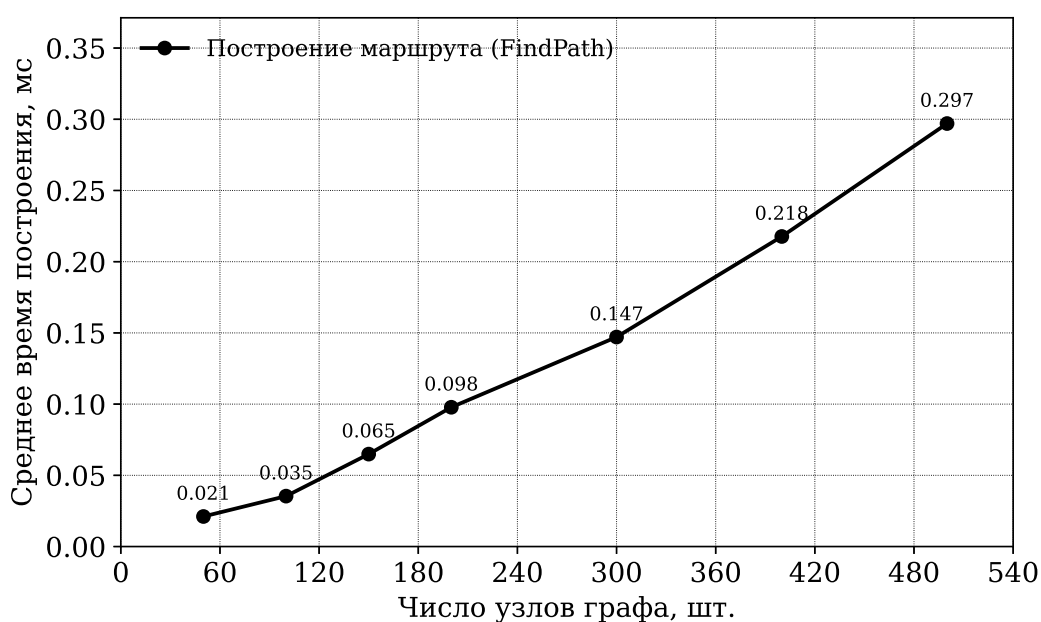


Рисунок 4.1 – Зависимость времени построения маршрута от размера графа

Время выполнения растёт практически линейно с увеличением размера графа, что согласуется с теоретической оценкой сложности $O((V + E) \log V)$ для разреженных графов [21]. Все измеренные значения находятся в субмиллисекундном диапазоне и укладываются в установленные требования с мно-

гократным запасом: даже на графе из 500 узлов построение маршрута занимает в среднем 0,297 мс при требовании менее 200 мс, а поиск ближайшего эвакуационного выхода – 0,630 мс при требовании менее 300 мс. Полученный запас по производительности подтверждает пригодность алгоритма Дейкстры для построения маршрутов в реальном времени и оставляет резерв для графов значительно большего объёма.

4.9 Системное тестирование

Системное тестирование выполнено для проверки совместной работы всех компонентов приложения IndoorNav в сборе на реальных данных навигационного графа здания (586 узлов, 1202 ребра) и на целевых платформах (Windows, iOS, Android). В отличие от модульного тестирования, проверяющего отдельные методы, системное тестирование оценивает сквозные сценарии «от действия пользователя до результата на экране» с участием всех слоёв архитектуры – представления, модели представления, сервисов и модели данных.

Ниже подробно описаны сквозные сценарии системного тестирования. Для каждого сценария приведены выполняемые действия и полученный результат, а для сценариев, проверенных в работающем приложении, – снимок экрана.

4.9.1 Построение маршрута между аудиториями

Сценарий. Пользователь выбирает начальную и конечную аудитории и запускает построение маршрута на реальном графе здания. Система загружает граф, выполняет поиск кратчайшего пути алгоритмом Дейкстры и формирует пошаговые инструкции.

Результат. Построен кратчайший маршрут между аудиториями, отображён на плане этажа; сформированы пошаговые инструкции. Результат соответствует требованию построения маршрута между точками и показан на рисунке 4.2.

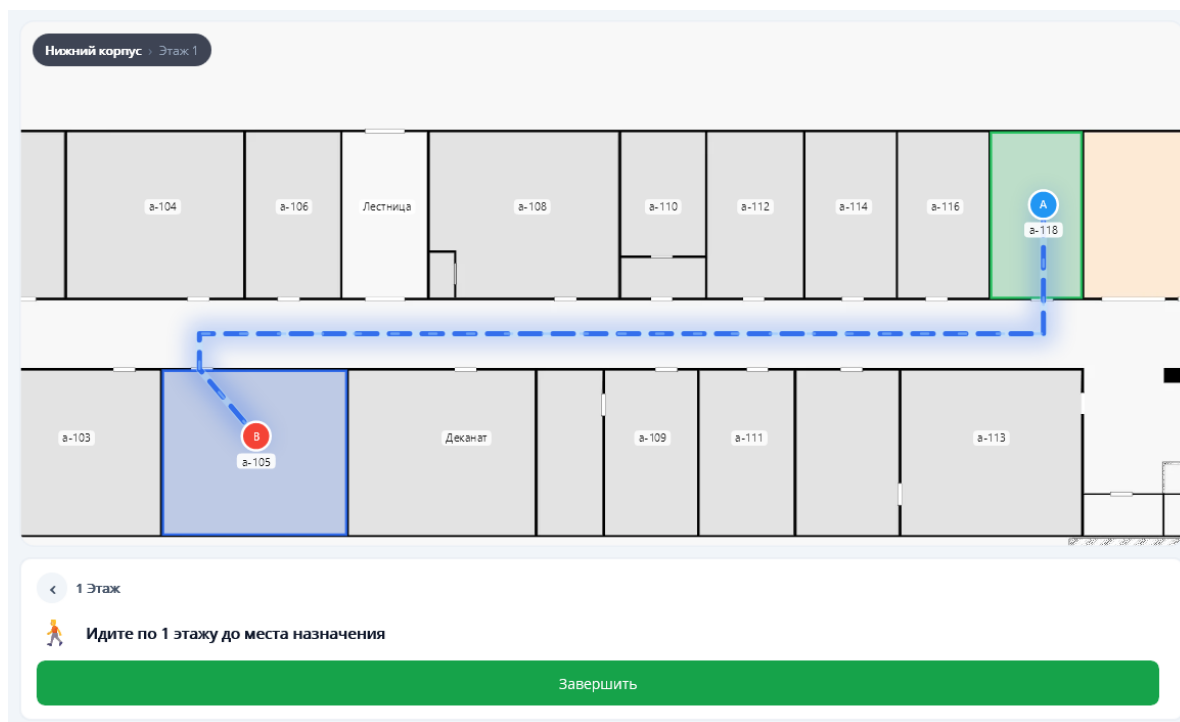


Рисунок 4.2 – Построение маршрута между аудиториями

4.9.2 Эвакуационный маршрут в режиме ЧС

Сценарий. Администратор активирует режим чрезвычайной ситуации, после чего система автоматически строит маршрут до ближайшего эвакуационного выхода от текущего положения пользователя.

Результат. Построен маршрут до ближайшего выхода. Результат соответствует требованию работы режима ЧС и показан на рисунке 4.3.

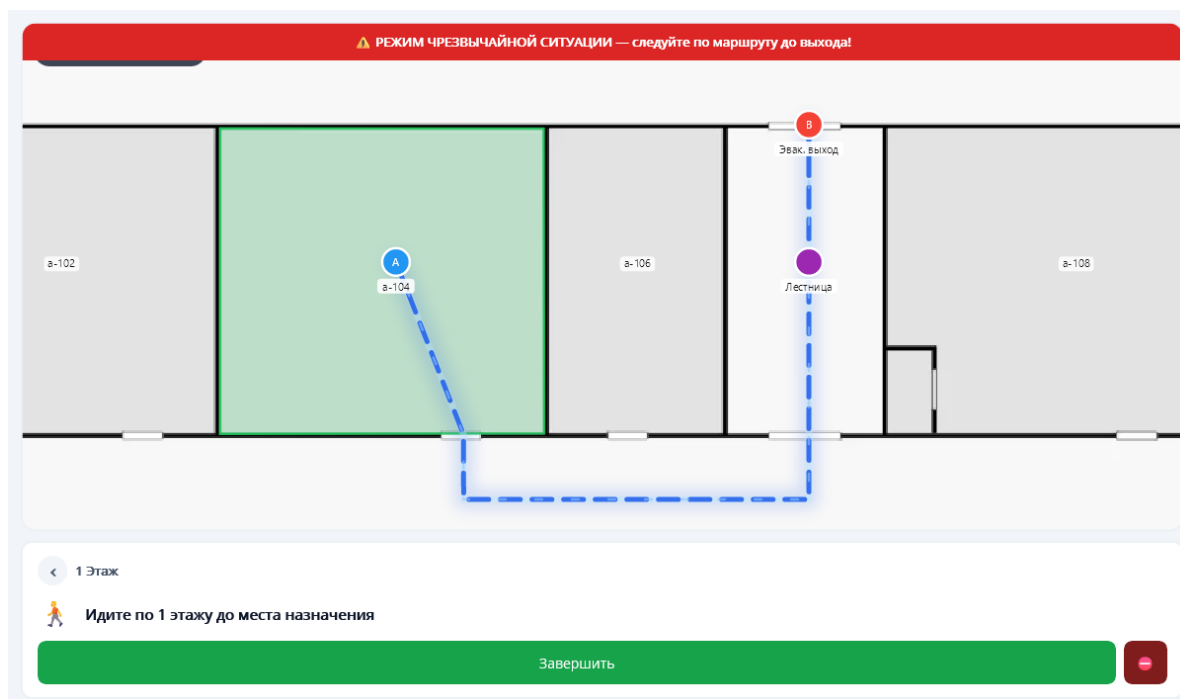


Рисунок 4.3 – Эвакуационный маршрут до ближайшего выхода в режиме ЧС

4.9.3 Перестроение маршрута при блокировке

Сценарий. Пользователь отмечает заблокированный узел на текущем маршруте, после чего система перестраивает путь.

Результат. Маршрут автоматически перестроен в обход заблокированного участка без перезагрузки графа. Результат подтверждает работу механизма динамического исключения узлов и показан на рисунке 4.4.

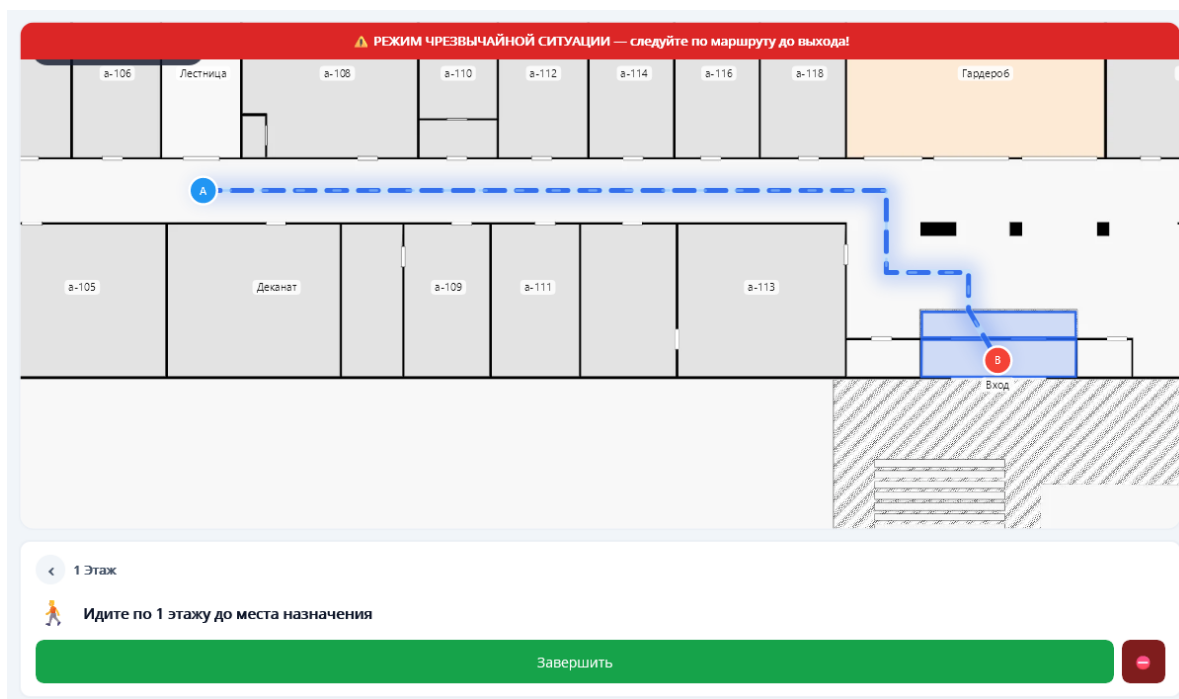


Рисунок 4.4 – Перестроение маршрута при блокировке участка

4.9.4 Позиционирование пользователя по QR-коду

Сценарий. Пользователь сканирует QR-код, размещённый на стене здания. Приложение разбирает ссылку вида `indoornav://node/{id}`, устанавливает соответствующий узел как начальную точку.

Результат. Начальная точка маршрута корректно определена по отсканированному QR-коду. Результат подтверждает интеграцию модуля разбора ссылок с алгоритмом маршрутизации и показан на рисунке 4.5.

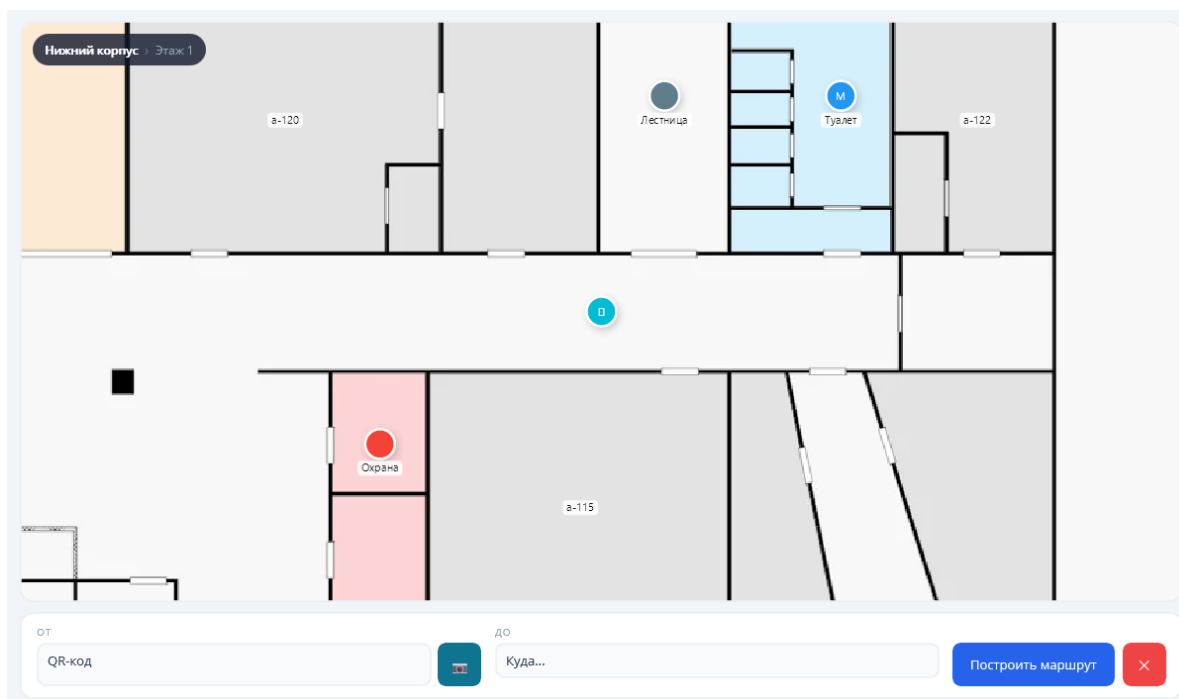


Рисунок 4.5 – Позиционирование пользователя по QR-коду

4.9.5 Определение исходной точки по расписанию в режиме ЧС

Сценарий. В здании активирован режим чрезвычайной ситуации. При входе студента в приложение система запрашивает подтверждение того, что он находится в аудитории согласно своему расписанию. При подтверждении эта аудитория автоматически устанавливается как исходная точка, и сразу строится маршрут до ближайшего эвакуационного выхода.

Результат. Исходная точка маршрута определена по подтверждённой аудитории из расписания без ручного выбора, эвакуационный маршрут до ближайшего выхода построен автоматически. Результат подтверждает совместную работу сервиса расписания и модуля эвакуационной маршрутизации и показан на рисунке 4.6.

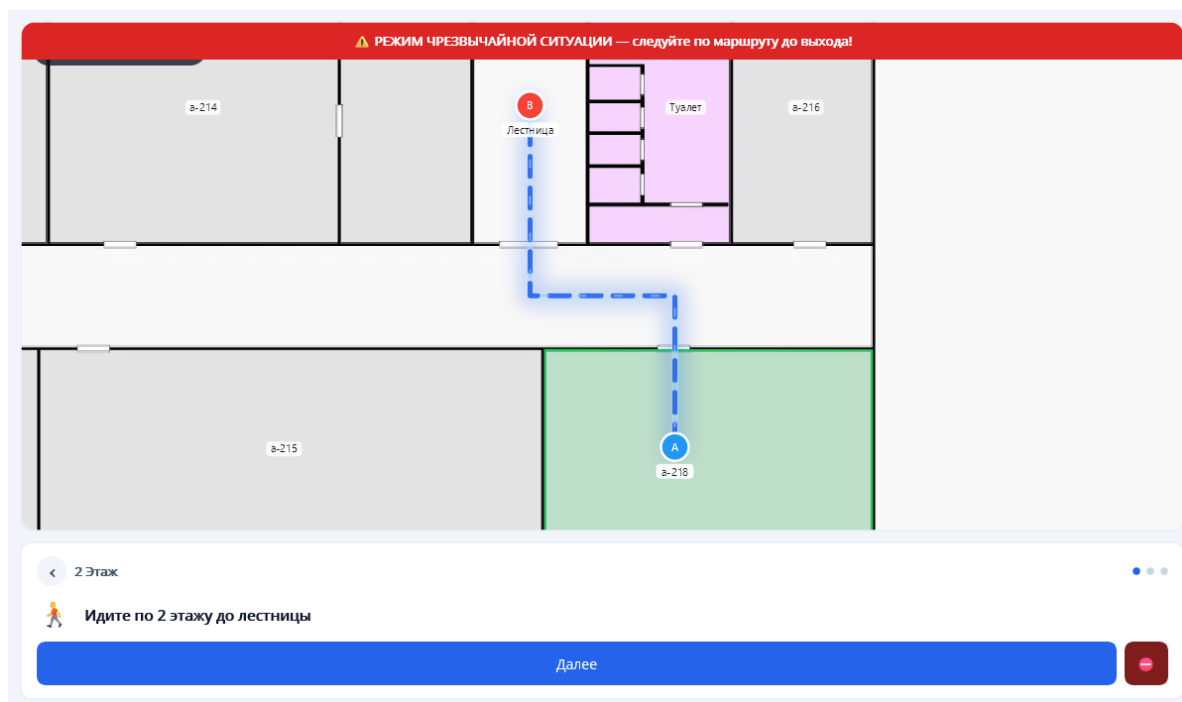


Рисунок 4.6 – Определение исходной точки по расписанию в режиме ЧС

Системное тестирование подтвердило корректную совместную работу всех слоёв приложения на реальных данных графа здания и на целевых платформах.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы выполнена программная реализация и тестирование алгоритмов динамической маршрутизации для системы построения эвакуационных маршрутов в замкнутых пространствах. Все поставленные задачи решены в полном объёме.

Проведён анализ существующих алгоритмов поиска кратчайших путей на графах. На основании сравнительного анализа по критериям оптимальности, производительности, поддержки динамического исключения узлов и сложности реализации выбран алгоритм Дейкстры как наилучшее решение для задачи эвакуационной маршрутизации. Алгоритм обеспечивает гарантию оптимальности при неотрицательных весах рёбер, эффективную работу на разреженных графах, характерных для планов зданий, и допускает модификацию для динамического исключения заблокированных узлов.

Разработаны структуры данных для представления навигационного графа: классы `NavGraph`, `NavNode` и `NavEdge`, обеспечивающие эффективное хранение и обработку топологии здания. Класс `NavNode` содержит полный набор атрибутов узла: координаты, номер этажа, идентификатор здания, флаги типа и группу перехода. Класс `NavEdge` хранит вес ребра с автоматическим вычислением евклидова расстояния для проходов. Реализована загрузка и сохранение графа в формате JSON средствами `System.Text.Json`.

Реализован алгоритм Дейкстры с модификациями для поддержки динамического исключения узлов. Множество исключённых узлов передаётся как параметр метода `FindPath` и проверяется при релаксации рёбер без модификации исходного графа. Конечный узел маршрута никогда не исключается. Алгоритм использует стандартные структуры данных .NET (`SortedSet`, `Dictionary`, `HashSet`) и обеспечивает время выполнения менее 200 мс для графов до 500 узлов, что соответствует требованиям реального времени. Оптимизация списка смежности и обработка устаревших записей очереди повышают эффективность реализации.

Разработан алгоритм поиска ближайшего эвакуационного выхода, учитывающий текущее положение пользователя, принадлежность к зданию и динамически изменяющиеся препятствия. Алгоритм приоритизирует выходы в текущем здании. Алгоритм интегрирован с сервисом экстренной эвакуации EmergencyService и ViewModel приложения IndoorNav.

Разработан механизм формирования множества исключённых узлов BuildExcludeSet, объединяющий заблокированные пользователем узлы и выходы, не являющиеся целевыми. Это предотвращает прохождение маршрута через другие выходы и обеспечивает корректное перестроение маршрута при блокировке участков.

Проведено комплексное тестирование алгоритмической части: модульное тестирование четырёх модулей маршрутизации (построение маршрута, поиск эвакуационного выхода, вычисление весов рёбер и формирование множества исключений), тестирование производительности на графах от 50 до 500 узлов, а также интеграционное тестирование на реальных данных графа здания университета. Результаты подтвердили корректность работы алгоритмов во всех сценариях, включая граничные случаи (пустой граф, несвязный граф, совпадение стартовой и конечной точек), и соответствие требованиям производительности.

Таким образом, в ходе выпускной квалификационной работы реализованы и верифицированы алгоритмы динамической маршрутизации для системы построения эвакуационных маршрутов в замкнутых пространствах. Полученные результаты могут быть применены в навигационных системах внутри зданий, приложениях экстренной эвакуации и иных программных комплексах, требующих эффективного построения маршрутов с учётом динамических ограничений. Дальнейшее развитие системы может включать реализацию алгоритма A^* с допустимой эвристикой, поддержку двунаправленного поиска и интеграцию с серверной частью для распределённого построения маршрутов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки. — URL: <https://docs.cntd.ru/document/1200007652> (дата обращения: 07.05.2026).
2. ГОСТ 19.105-78. Единая система программной документации. Общие требования к программным документам. — URL: <https://docs.cntd.ru/document/1200007655> (дата обращения: 07.05.2026).
3. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — URL: <https://docs.cntd.ru/document/1200007659> (дата обращения: 07.05.2026).
4. ГОСТ 19.301-79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. — URL: <https://docs.cntd.ru/document/1200007663> (дата обращения: 07.05.2026).
5. ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов. — URL: <https://docs.cntd.ru/document/1200121069> (дата обращения: 07.05.2026).
6. Федеральный закон от 22.07.2008 № 123-ФЗ «Технический регламент о требованиях пожарной безопасности». — URL: https://www.consultant.ru/document/cons_doc_LAW_78699/ (дата обращения: 07.05.2026).
7. СП 1.13130.2009. Системы противопожарной защиты. Эвакуационные пути и выходы. — URL: <https://docs.cntd.ru/document/1200071143> (дата обращения: 07.05.2026).
8. Ecma International. The JSON Data Interchange Format. Standard ECMA-404, 2nd ed. — URL: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/> (дата обращения: 07.05.2026).
9. Microsoft. .NET Multi-platform App UI (MAUI) documentation. — URL: <https://learn.microsoft.com/dotnet/maui/> (дата обращения: 07.05.2026).

10. Microsoft. Model-View-ViewModel (MVVM) in .NET MAUI. — URL: <https://learn.microsoft.com/dotnet/maui/xaml/fundamentals/mvvm> (дата обращения: 07.05.2026).
11. Microsoft. Dependency injection in .NET MAUI. — URL: <https://learn.microsoft.com/dotnet/maui/fundamentals/dependency-injection> (дата обращения: 07.05.2026).
12. Microsoft. XAML overview in .NET MAUI. — URL: <https://learn.microsoft.com/dotnet/maui/xaml/> (дата обращения: 07.05.2026).
13. Microsoft. SkiaSharp graphics in .NET MAUI. — URL: <https://learn.microsoft.com/dotnet/maui/user-interface/graphics/skiasharp/> (дата обращения: 07.05.2026).
14. Microsoft. System.Text.Json overview. — URL: <https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/overview> (дата обращения: 07.05.2026).
15. Navigine. Indoor Navigation and Positioning SDK Documentation. — URL: <https://navigine.com/sdk-documentation/> (дата обращения: 07.05.2026).
16. Situm Technologies. Situm Indoor Positioning Platform. Developer Documentation. — URL: <https://situm.com/docs/> (дата обращения: 07.05.2026).
17. Mappedin Inc. Mappedin Developer Documentation. — URL: <https://developer.mappedin.com/docs/> (дата обращения: 07.05.2026).
18. 2ГИС. Поэтажные планы в MapGL JavaScript API. — URL: <https://docs.2gis.com/ru/mapgl/reference/indoor> (дата обращения: 07.05.2026).
19. Mapsted Corp. Indoor Navigation Technology. — URL: <https://mapsted.com/indoor-navigation/> (дата обращения: 07.05.2026).
20. Амеличев Г.Э., Панина В.С. Сравнительный анализ алгоритмов поиска пути в помещении // E-Scio. — 2022. — URL: <https://cyberleninka.ru/article/n/sravnitelnyy-analiz-algoritmov-poiska-puti-v-pomeschenii> (дата обращения: 07.05.2026).
21. Близнякова Е.А., Куликов А.А., Куликов А.В. Сравнительный анализ методов поиска кратчайшего пути в графе // Архитектура, строительство,

транспорт. — 2022. — Т. 1 (99). — С. 80–87. — DOI: 10.31660/2782-232X-2022-1-80-87.

22. Винников М.Д. Разработка базы данных для мобильного приложения на операционной системе Android // Научные известия. — 2022. — URL: <https://cyberleninka.ru/article/n/razrabotka-bazy-dannyh-dlya-mobilnogo-prilozheniya-na-operatsionnoy-sisteme-android> (дата обращения: 07.05.2026).

23. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Паттерны объектно-ориентированного проектирования / пер. с англ. — СПб. : Питер, 2021. — 448 с. — ISBN 978-5-4461-1595-2.

24. Гравит М.В., Карькин И.Н., Дмитриев И.И., Кузенков К.А. Моделирование процесса эвакуации из высотных зданий и сооружений с использованием пассажирских лифтов // Пожаровзрывобезопасность. — 2019. — Т. 28, № 2. — С. 66–80. — DOI: 10.18322/PVB.2019.28.02.66-80.

25. Гриняк В.М., Гриняк Т.М., Цибанов П.А. Позиционирование внутри помещений с помощью Bluetooth-устройств // Территория новых возможностей. Вестник ВГУЭС. — 2018. — Т. 10, № 2. — С. 137–147. — DOI: 10.24866/VVSU/2073-3984/2018-2/137-147.

26. Гусарова А.А. Современные информационные технологии как инструмент повышения пожарной безопасности зданий // Пожаровзрывобезопасность. — 2023. — Т. 32, № 6. — С. 36–46. — DOI: 10.22227/0869-7493.2023.32.06.36-46.

27. Гюллинг А.О., Воронцова Н.В. Визуализация алгоритма Дейкстры // Известия Тульского государственного университета. Технические науки. — 2024. — Т. 2. — С. 127–128. — DOI: 10.24412/2071-6168-2024-2-127-128.

28. Ершов Т.А. Обзор технологии мобильной разработки Xamarin // StudNet. — 2021. — URL: <https://cyberleninka.ru/article/n/obzor-tehnologii-mobilnoy-razrabotki-xamarin> (дата обращения: 07.05.2026).

29. Ескендир М.А. Введение в разработку мобильных приложений // Вестник магистратуры. — 2019. — URL: <https://cyberleninka.ru/article/n/vvedenie-v-razrabotku-mobilnyh-prilozheniy> (дата обращения: 07.05.2026).

30. Зайцева И.А., Эбата В.Ч., Ковбаса Н.А. Практика применения методологий Agile, Scrum в ИТ-проектах // Индустриальная экономика. — 2021. — Т. 1. — DOI: 10.47576/2712-7559_2021_1_62.
31. Жуковская А.Н., Заушицина А.С. Особенности разработки кроссплатформенных мобильных приложений // Решетнёвские чтения. — 2017. — URL: <https://cyberleninka.ru/article/n/osobennosti-razrabotki-krossplatformennyh-mobilnyh-prilozheniy> (дата обращения: 07.05.2026).
32. Изотова Т.Ю. Обзор алгоритмов поиска кратчайшего пути в графе // Новые информационные технологии в автоматизированных системах. — 2016. — URL: <https://cyberleninka.ru/article/n/obzor-algoritmov-poiska-kratchayshego-puti-v-grafe> (дата обращения: 07.05.2026).
33. Ильичёв М.В., Мезенцева Е.М. Фреймворк Xamarin // Теория и практика современной науки. — 2018. — URL: <https://cyberleninka.ru/article/n/freymvork-xamarin> (дата обращения: 07.05.2026).
34. Исаев М.И., Вахаева Д.А., Алдамов А.И. Алгоритмы для нахождения кратчайшего пути // Региональная и отраслевая экономика. — 2023. — № 4. — С. 16–19. — DOI: 10.47576/2949-1916_2023_4_16.
35. Карманова М.В. Концепция разработки мобильных приложений для картографирования зон чрезвычайных ситуаций и происшествий // Интерэкспо Гео-Сибирь. — 2020. — Т. 1, № 2. — С. 68–74. — DOI: 10.33764/2618-981X-2020-1-2-68-74.
36. Кошелев О.В. Шаблоны проектирования в программировании // Научный журнал. — 2018. — URL: <https://cyberleninka.ru/article/n/shablony-proektirovaniya-v-programmirovanii> (дата обращения: 07.05.2026).
37. Куликов А.С. Обзор основных методов внутренней навигации // Форум молодых учёных. — 2021. — URL: <https://cyberleninka.ru/article/n/obzor-osnovnyh-metodov-vnutrenniy-navigatsii> (дата обращения: 07.05.2026).
38. Маринин А.К. Выбор архитектуры для мобильных приложений // Известия Кабардино-Балкарского НЦ РАН. — 2024. — Т. 26, № 5. — С. 84–93. — DOI: 10.35330/1991-6639-2024-26-5-84-93.

39. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения / пер. с англ. — СПб. : Питер, 2022. — 352 с. — ISBN 978-5-4461-0772-8.

40. Монгуш А.В., Кикин П.М. Обзор технологий indoor-навигации // Интерэкспо Гео-Сибирь. — 2017. — URL: <https://cyberleninka.ru/article/n/obzor-tehnologiy-indoor-navigatsii> (дата обращения: 07.05.2026).

41. Наумов Р.К., Железков Н.Э. Сравнительный анализ форматов хранения текстовых данных для дальнейшей обработки методами машинного обучения // Научный результат. Информационные технологии. — 2021. — Т. 6, № 1. — С. 40–47. — DOI: 10.18413/2518-1092-2021-6-1-0-5.

42. Охотиченко А.В., Ухта Ю.Б. Проектирование системы для навигации внутри здания со сложной иерархической структурой // Программные системы и вычислительные методы. — 2021. — № 4. — С. 46–57. — DOI: 10.7256/2454-0714.2021.4.37012.

43. Сазонов А.П. Использование принципа Dependency Injection в мобильной разработке // Инновации и инвестиции. — 2024. — URL: <https://cyberleninka.ru/article/n/ispolzovanie-printsipa-dependency-injection-v-mobilnoy-razrabotke-na-territorii-rb> (дата обращения: 07.05.2026).

44. Самошин Д.А., Георгиев Н.А., Климовцова Я.О., Кочетыгов В.А. Затраты времени на изучение плана эвакуации и рекомендации по повышению его эффективности // Пожары и чрезвычайные ситуации: предотвращение, ликвидация. — 2026. — № 1. — С. 26–36. — DOI: 10.25257/FE.2026.1.26-36.

45. Сергеев К.С., Афонин А.Ю., Осин С.М. Особенности и различия архитектурных паттернов MV в языке программирования C# // Парадигма. — 2025. — № 4. — Ч. 2. — URL: <https://cyberleninka.ru/article/n/osobennosti-i-razlichiya-arhitekturnyh-patternov-mv-v-yazyke-programmirovaniya-s> (дата обращения: 07.05.2026).

46. Стерлягов С.П., Селютина Е.П. Применение User Experience/User Interface моделирования для разработки мобильного приложения // Меж-

дународный научно-исследовательский журнал. — 2017. — Т. 62. — DOI: 10.23670/IRJ.2017.62.067.

47. Ткачев Д.В. Графы и их применение при чрезвычайных ситуациях // Вестник науки и образования. — 2019. — URL: <https://cyberleninka.ru/article/n/grafy-i-ih-primenenie-pri-chrezvychaynyh-situatsiyah> (дата обращения: 07.05.2026).

48. Троелсен Э., Джепикс Ф. Язык программирования C# 7 и платформы .NET и .NET Core / пер. с англ. — М. : Диалектика, 2018. — 1328 с. — ISBN 978-5-6040723-1-8.

49. Уваров А.Н. Инверсия управления и внедрение зависимостей // Символ науки. — 2016. — № 10-1. — URL: <https://cyberleninka.ru/article/n/inversiya-upravleniya-i-vnedrenie-zavisimostey> (дата обращения: 07.05.2026).

50. Урбанович М.В., Ковалева К.А. Принципы разработки пользовательских интерфейсов // Современные инновации, системы и технологии. — 2023. — Т. 3, № 4. — DOI: 10.47813/2782-2818-2023-3-4-0363-0374.

51. Фешина Е.В., Пчела Д.В., Долгов А.В., Трофимов Д.И. Анализ технологий разработки мобильных приложений и информационных систем на базе ОС Android // Вестник Адыгейского государственного университета. Сер. 4. — 2022. — Т. 1 (296). — С. 85–91. — DOI: 10.53598/2410-3225-2022-1-296-85-91.

52. Яровая Е.В. Принципы построения архитектуры программного обеспечения // E-Scio. — 2022. — URL: <https://cyberleninka.ru/article/n/printsipy-postroeniya-arhitektury-programmnogo-obespecheniya> (дата обращения: 07.05.2026).

53. Deng H., Ou Z., Zhang G., Deng Y., Tian M. BIM and Computer Vision-Based Framework for Fire Emergency Evacuation Considering Local Safety Performance // Sensors. — 2021. — Vol. 21, No. 11. — Art. 3851. — DOI: 10.3390/s21113851.

54. Han L., Guo H., Zhang H., Kong Q., Zhang A., Gong C. An Efficient Staged Evacuation Planning Algorithm Applied to Multi-Exit Buildings // ISPRS International Journal of Geo-Information. — 2020. — Vol. 9, No. 1. — Art. 46. — DOI: 10.3390/ijgi9010046.

55. Jia X., Ebone A., Tan Y. A Performance Evaluation of Cross-Platform Mobile Application Development Approaches // Proceedings of the 5th IEEE/ACM International Conference on Mobile Software Engineering and Systems (MOBILESoft 2018). — Gothenburg, 2018. — P. 92–93. — DOI: 10.1145/3197231.3197252.
56. Landau Y., Ben-Moshe B. STEPS: An Indoor Navigation Framework for Mobile Devices // Sensors. — 2020. — Vol. 20, No. 14. — Art. 3929. — DOI: 10.3390/s20143929.
57. Mirahadi F., McCabe B.Y. EvacuSafe: Building Evacuation Strategy Selection Using Route Risk Index // Journal of Computing in Civil Engineering (ASCE). — 2020. — Vol. 34, No. 2. — Art. 04019051. — DOI: 10.1061/(ASCE)CP.1943-5487.0000867.
58. Mirahadi F., McCabe B.Y. EvacuSafe: A Real-Time Model for Building Evacuation Based on Dijkstra's Algorithm // Journal of Building Engineering. — 2021. — Vol. 34. — Art. 101687. — DOI: 10.1016/j.jobbe.2020.101687.
59. Rubio-Sandoval J.I., Martínez-Rodríguez J.-L., Lopez-Arevalo I., Ríos-Alvarado A.B., Rodríguez-Rodríguez A.J., Vargas-Requena D.T. An Indoor Navigation Methodology for Mobile Devices by Integrating Augmented Reality and Semantic Web // Sensors. — 2021. — Vol. 21, No. 16. — Art. 5435. — DOI: 10.3390/s21165435.
60. Singh N., Choe S., Punmiya R. Machine Learning Based Indoor Localization Using Wi-Fi RSSI Fingerprints: An Overview // IEEE Access. — 2021. — Vol. 9. — P. 127150–127174. — DOI: 10.1109/ACCESS.2021.3111083.
61. Tomažič S. Indoor Positioning and Navigation // Sensors. — 2021. — Vol. 21, No. 14. — Art. 4793. — DOI: 10.3390/s21144793.
62. Yen H.-H., Lin C.-H., Tsao H.-W. Time-Aware and Temperature-Aware Fire Evacuation Path Algorithm in IoT-Enabled Multi-Story Multi-Exit Buildings // Sensors. — 2021. — Vol. 21, No. 1. — Art. 111. — DOI: 10.3390/s21010111.
63. Zhang J., Guo J., Xiong H., Liu X., Zhang D. A Framework for an Intelligent and Personalized Fire Evacuation Management System // Sensors. — 2019. — Vol. 19, No. 14. — Art. 3128. — DOI: 10.3390/s19143128.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.8.

Сведения о ВКРБ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

«Бизнес-проект «Программная реализация системы эвакуации маршрутов для замкнутых пространств.» Программная реализация и тестирование алгоритмов динамической маршрутизации»

Руководитель ВКРБ
д.т.н, профессор
Томакова Римма Александровна

Автор ВКРБ
студент группы ПО-216
Старков Вадим Артемович

				ВКРБ 2206111.09.03.04.26.019			
Инициалы Фамилия Имя Отчество	Фамилия И.О.	Инициалы	Имя	Сведения о ВКРБ			
	Старков В.А.	Инициалы	В.А.	Лист 1			
	Томакова Р.А.	Инициалы	Р.А.	Лист 6			
				Выпускная квалификационная работа бакалавра			
				ОЗГУ ПО-216			

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Целью работы является программная реализация системы построения эвакуационных маршрутов в реальном времени для замкнутых пространств.

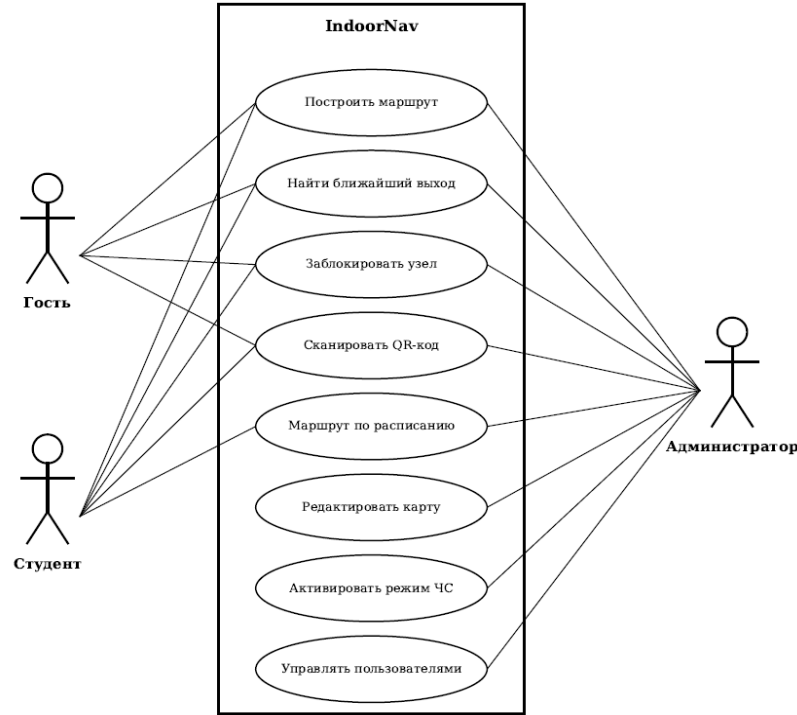
Для достижения цели необходимо решить следующие задачи:

- рассмотреть известные алгоритмы поиска кратчайших путей на графах
- спроектировать структуры данных для хранения и обработки навигационного графа
- реализовать алгоритм Дейкстры с поддержкой динамического исключения узлов
- реализовать поиск ближайшего эвакуационного выхода от текущего положения пользователя
- построить механизм формирования множества исключаемых узлов для перестроения маршрута
- провести модульное и системное тестирование алгоритмов на разных сценариях
- оценить быстродействие и правильность работы алгоритмов.

ВКРБ 2206111.09.03.04.26.019			
Имя работы	Фамилия И.О.	Инициалы	Имя
Учебный год	Семестр	Год	Мес
Специальность	Специальность	Год	Мес
Цель и задачи разработки			
Выполнен и одобрен руководителем			
03.09.2025			

Рисунок А.2 – Цель и задачи разработки

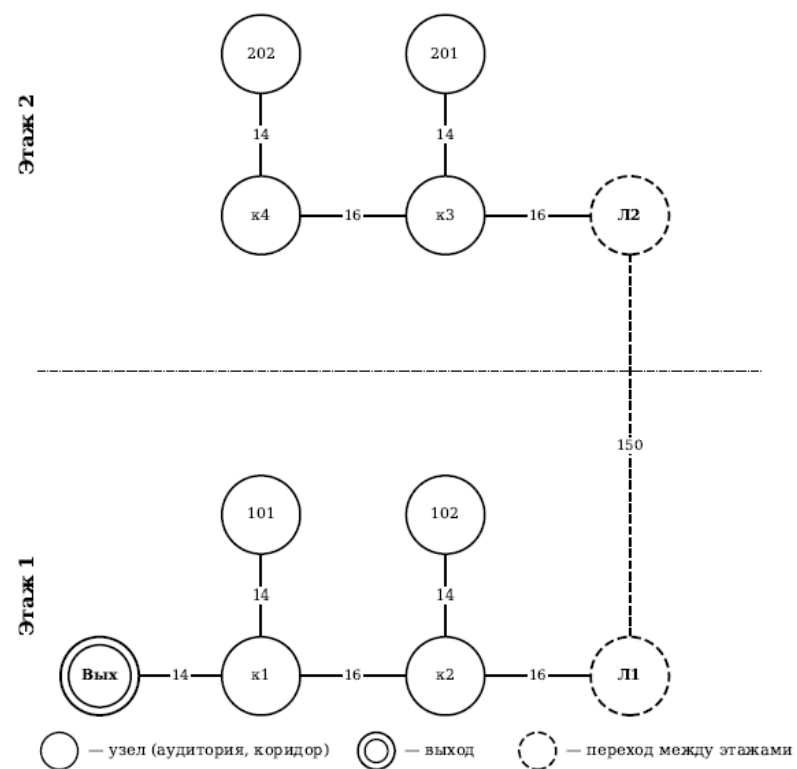
Диаграмма вариантов использования



			ВКРБ 2206111.09.03.04.26.019			
			Дизайнерка вариантов использования	Лит	Нач	Нес
Имя работы	Фамилия И.О.	Имя				
Руководитель	Степанов И.А.	Иванов				
Специалист	Петров П.А.	Петров				
			Выпускная квалификационная работа бакалавра	Лист 3		Листов 8
				03/04 ПО-216		

Рисунок А.3 – Диаграмма вариантов использования

Концептуальная модель программы



ВКРБ 2206111.09.03.04.26.019			
Имя работ Фамилия Инициалы	Фамилия И.О.	Инициалы	Имя
	Инициалы И.О.	Инициалы И.О.	Инициалы И.О.
	Инициалы И.О.	Инициалы И.О.	Инициалы И.О.
Концептуальная модель программы			
Выпускная квалификационная работа бакалавра			
03/19 ПО-216			

Рисунок А.4 – Концептуальная модель программы

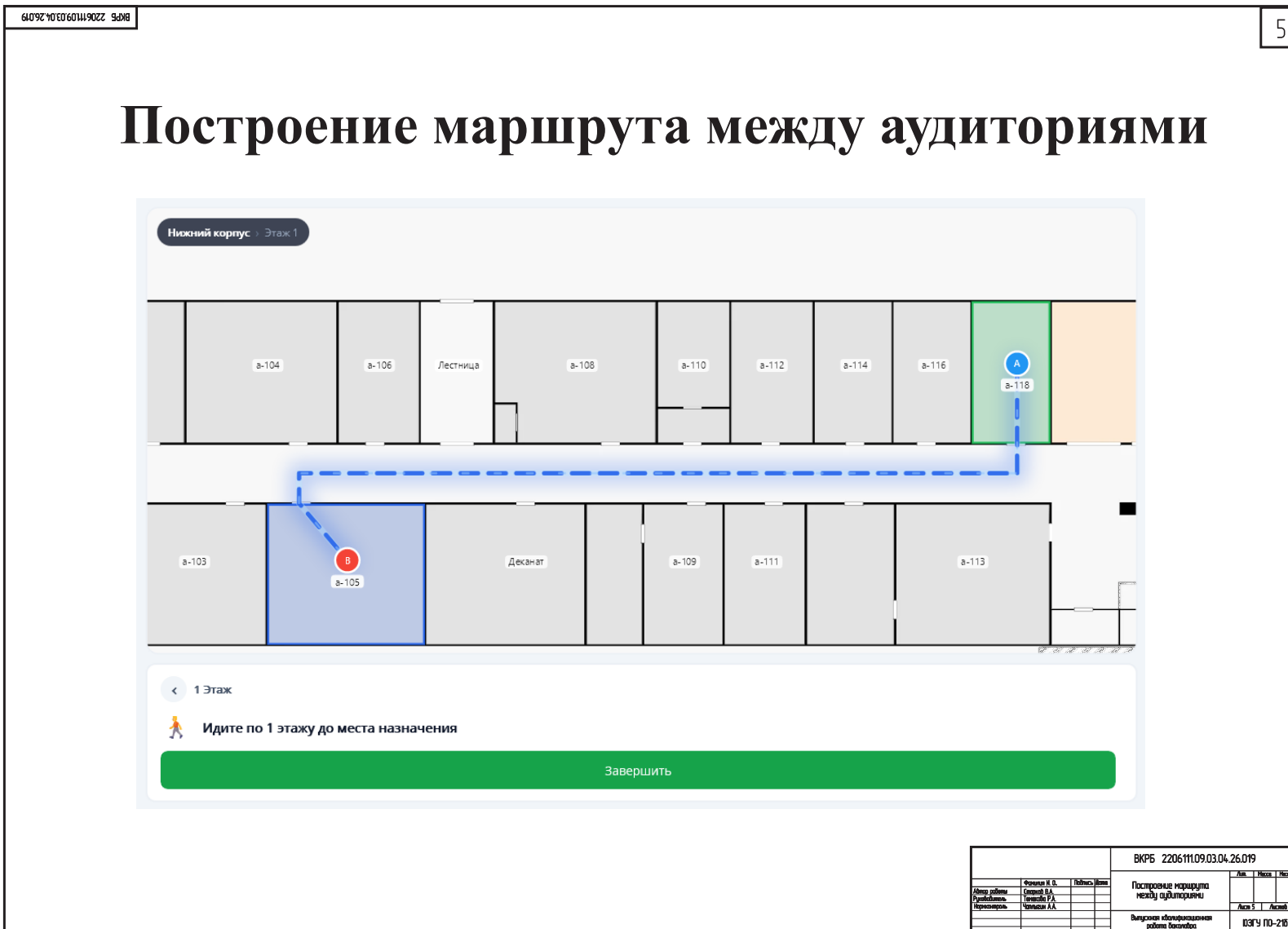
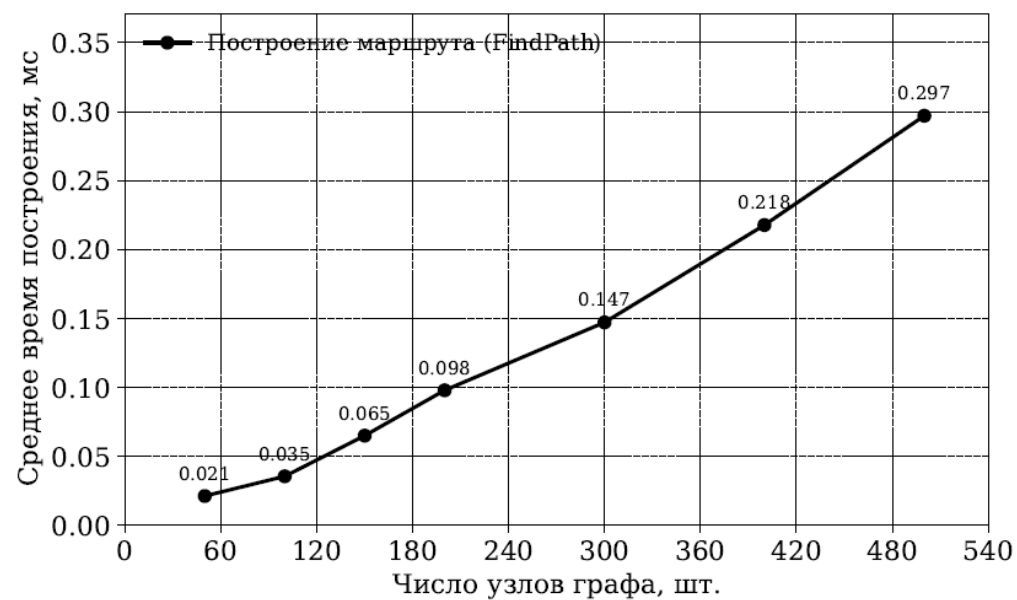


Рисунок А.5 – Построение маршрута между аудиториями

Время построения маршрута в зависимости от размера графа



ВКРБ 2206111.09.03.04.26.019			
Имя работы	Фамилия И.О.	Инициалы	Дата
Фамилия	Инициалы	Инициалы	Дата
Инициалы	Инициалы	Инициалы	Дата
Время построения маршрута в зависимости от размера графа			Лист 6
Выполнен идентификацией работы			Лист 6
Инициалы			03.04.2016

Рисунок А.6 – Время построения маршрута в зависимости от размера графа

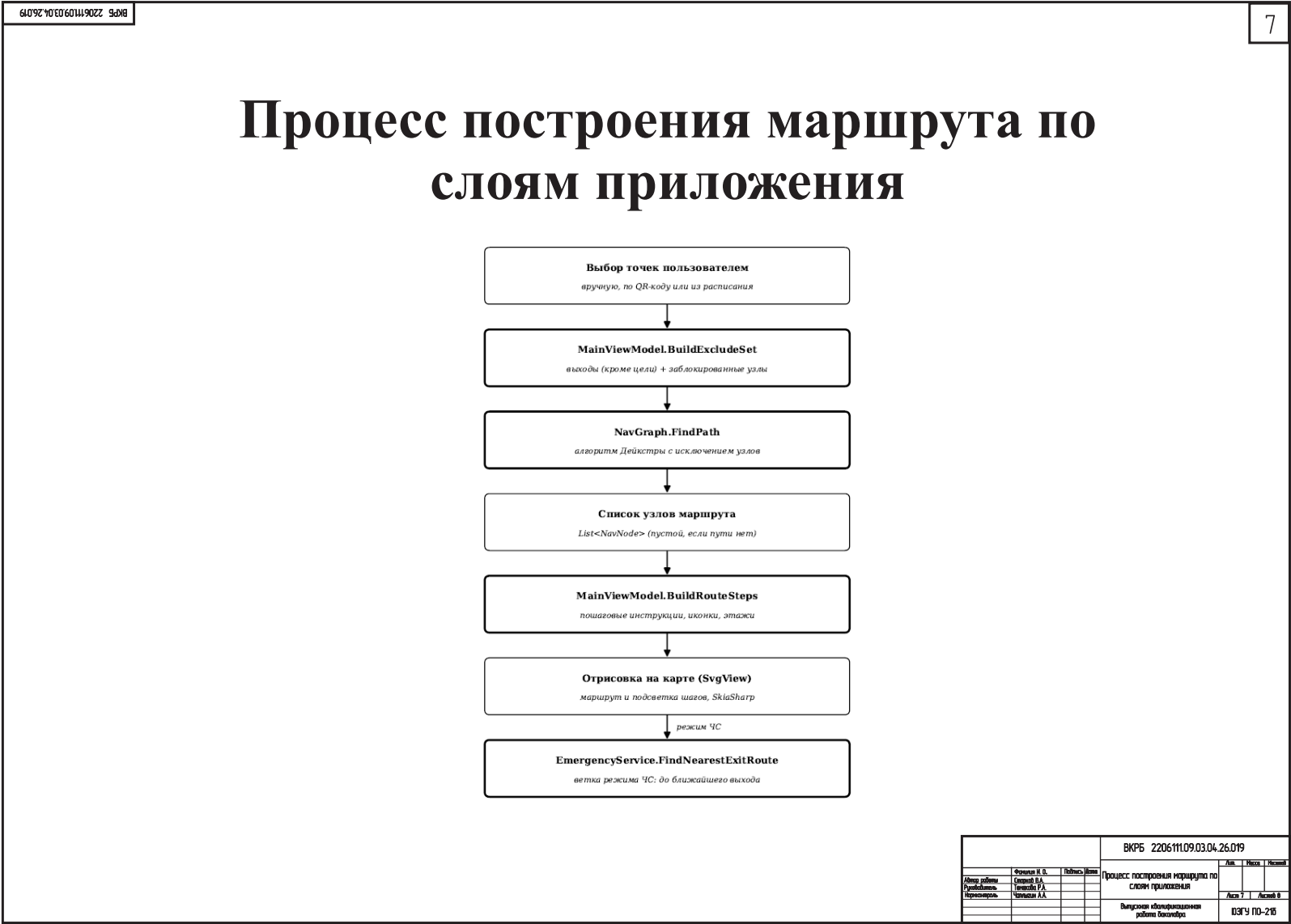


Рисунок А.7 – Процесс построения маршрута по слоям приложения

Основные результаты работы:

- Все требования, объявленные в техническом задании, были полностью выполнены, все задачи, поставленные в начале разработки, успешно решены.

		БКРБ 2206111.09.03.04.26.019	
		Заключение	Лит. Искл. И
			Лит. В. / Искл.
Автор работы	Ференц Н. В.	Высказан и мотивирован работой Стороженко.	ИЗГУ ПО-21
Руководитель	Семаров В. А.		
Перепроверяющий	Семаров Р. А.		
	Чиркина А. А.		

Рисунок А.8 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

Models/NavNode.cs

```
1 namespace IndoorNav.Models;
2
3 public class NavNode
4 {
5     public string Id { get; set; } = Guid.NewGuid().ToString();
6     public string Name { get; set; } = string.Empty;
7
8     public string BuildingId { get; set; } = string.Empty;
9     public int FloorNumber { get; set; }
10
11     public float X { get; set; }
12     public float Y { get; set; }
13
14     public bool IsTransition { get; set; }
15     public bool IsElevator { get; set; }
16     public string TransitionGroupId { get; set; } = string.Empty;
17
18     public bool IsQrAnchor { get; set; }
19     public bool IsExit { get; set; }
20     public bool IsFireExtinguisher { get; set; }
21     public bool IsEvacuationExit { get; set; }
22     public bool IsHidden { get; set; }
23     public bool IsLabelHidden { get; set; }
24
25     public float NodeRadiusScale { get; set; } = 1f;
26     public float LabelScale { get; set; } = 1f;
27     public string? NodeColorHex { get; set; }
28
29     public bool IsRoom { get; set; }
30     public bool IsWaypoint { get; set; }
31     public string InnerLabel { get; set; } = string.Empty;
32     public string SearchTags { get; set; } = string.Empty;
33     public string Category { get; set; } = string.Empty;
34     public bool IsCategoryHidden { get; set; }
35
36     public string DisplayName => string.IsNullOrEmpty(SearchTags)
37         ? Name
38         : $"{Name} {SearchTags}";
39
40     public List<List<float[]>>? Boundaries { get; set; }
41
42     public override string ToString() => Name;
43 }
```

Models/NavEdge.cs

```
1 namespace IndoorNav.Models;
2
3 public class NavEdge
```

```

4 {
5     public string FromId { get; set; } = string.Empty;
6     public string ToId { get; set; } = string.Empty;
7
8     public float Weight { get; set; }
9
10    public bool IsCrossFloor { get; set; }
11 }

```

Models/NavGraph.cs

```

1 namespace IndoorNav.Models;
2
3 public class NavGraph
4 {
5     public List<NavNode> Nodes { get; set; } = new();
6     public List<NavEdge> Edges { get; set; } = new();
7
8     public NavNode? GetNode(string id) => Nodes.FirstOrDefault(n => n.Id ==
9         id);
10
11     public IEnumerable<NavNode> GetNodesForFloor(string buildingId, int floor
12         ) =>
13         Nodes.Where(n => n.BuildingId == buildingId && n.FloorNumber == floor
14             );
15
16     public IEnumerable<NavEdge> GetEdgesForFloor(string buildingId, int floor
17         ) =>
18         Edges.Where(e =>
19             {
20                 var from = GetNode(e.FromId);
21                 var to = GetNode(e.ToId);
22                 return from != null && to != null
23                     && from.BuildingId == buildingId && from.FloorNumber == floor
24                     && to.BuildingId == buildingId && to.FloorNumber == floor
25                     ;
26             });
27
28     public void AddEdge(string fromId, string toId, bool crossFloor = false)
29     {
30         var from = GetNode(fromId);
31         var to = GetNode(toId);
32         if (from == null || to == null) return;
33
34         float weight = crossFloor ? 150f
35             : MathF.Sqrt(MathF.Pow(to.X - from.X, 2) + MathF.Pow(to.Y - from.
36                 Y, 2));
37
38         if (!Edges.Any(e => e.FromId == fromId && e.ToId == toId))
39             Edges.Add(new NavEdge { FromId = fromId, ToId = toId, Weight =
40                 weight, IsCrossFloor = crossFloor });
41
42         if (!Edges.Any(e => e.FromId == toId && e.ToId == fromId))
43             Edges.Add(new NavEdge { FromId = toId, ToId = fromId, Weight =
44                 weight, IsCrossFloor = crossFloor });
45     }
46 }

```

```

37     }
38
39     public void RemoveEdge(string fromId, string toId)
40     {
41         Edges.RemoveAll(e =>
42             (e.FromId == fromId && e.ToId == toId) ||
43             (e.FromId == toId && e.ToId == fromId));
44     }
45
46     public bool EdgeExists(string fromId, string toId) =>
47         Edges.Any(e => (e.FromId == fromId && e.ToId == toId) ||
48             (e.FromId == toId && e.ToId == fromId));
49
50     public List<NavNode> FindPath(string startId, string endId)
51         => FindPath(startId, endId, null);
52
53     public List<NavNode> FindPath(string startId, string endId, ISet<string>?
54         excludeIds)
55     {
56         if (startId == endId)
57             return GetNode(startId) is { } n ? [n] : [];
58
59         var dist = new Dictionary<string, float>();
60         var prev = new Dictionary<string, string?>();
61         var queue = new SortedSet<(float dist, string id)>(
62             Comparer<(float, string)>.Create((a, b) =>
63                 a.Item1 != b.Item1 ? a.Item1.CompareTo(b.Item1)
64                 : string.Compare(a.Item2, b.Item2, StringComparison.
65                     Ordinal)));
66
67         foreach (var node in Nodes)
68         {
69             dist[node.Id] = float.MaxValue;
70             prev[node.Id] = null;
71         }
72         dist[startId] = 0;
73         queue.Add((0, startId));
74
75         var adj = Nodes.ToDictionary(n => n.Id, _ => new List<(string
76             neighbor, float weight)>());
77         foreach (var edge in Edges)
78         {
79             if (adj.ContainsKey(edge.FromId))
80                 adj[edge.FromId].Add((edge.ToId, edge.Weight));
81         }
82
83         while (queue.Count > 0)
84         {
85             var (d, u) = queue.Min;
86             queue.Remove(queue.Min);
87
88             if (u == endId) break;
89             if (d > dist[u]) continue;

```



```

87
88         foreach (var (v, w) in adj[u])
89         {
90             if (excludeIds != null && excludeIds.Contains(v) && v !=
91                 endId) continue;
92             float alt = dist[u] + w;
93             if (alt < dist[v])
94             {
95                 queue.Remove((dist[v], v));
96                 dist[v] = alt;
97                 prev[v] = u;
98                 queue.Add((alt, v));
99             }
100         }
101
102         if (dist[endId] == float.MaxValue) return [];
103
104         var path = new List<NavNode>();
105         var current = endId;
106         while (current != null)
107         {
108             var node = GetNode(current);
109             if (node != null) path.Insert(0, node);
110             prev.TryGetValue(current, out var p);
111             current = p!;
112         }
113         return path;
114     }
115 }

```

Services/EmergencyService.cs

```

1 using System.Text.Json;
2 using IndoorNav.Models;
3
4 namespace IndoorNav.Services;
5
6 public class EmergencyChangedEventArgs : EventArgs
7 {
8     public string? BuildingId { get; init; }
9     public bool IsActive { get; init; }
10 }
11
12 public class EmergencyService
13 {
14     public event EventHandler<EmergencyChangedEventArgs>? EmergencyChanged;
15
16     private readonly HashSet<string> _activeBuildings = new();
17
18     private static string GetProjectRootPath()
19     {
20         var basePath = AppContext.BaseDirectory;
21         var dir = new DirectoryInfo(basePath);

```

```

22         while (dir != null && !File.Exists(Path.Combine(dir.FullName, "
                IndoorNav.csproj")))
23             dir = dir.Parent;
24         return dir?.FullName ?? basePath;
25     }
26
27     private static string StatePath =>
28         Path.Combine(GetProjectRootPath(), "Resources", "Raw", "
                emergency_state.json");
29
30     public bool IsEmergencyActive => _activeBuildings.Count > 0;
31
32     public bool IsActiveForBuilding(string? buildingId) =>
33         !string.IsNullOrEmpty(buildingId) && _activeBuildings.Contains(
            buildingId);
34
35     public string EmergencyMessage => IsEmergencyActive
36         ? "РЕЖИМ ЧРЕЗВЫЧАЙНОЙ СИТУАЦИИ - следуйте по маршруту до выхода!"
37         : string.Empty;
38
39     public async Task LoadAsync()
40     {
41         try
42         {
43             if (!File.Exists(StatePath)) return;
44             var json = await File.ReadAllTextAsync(StatePath);
45             var ids = JsonSerializer.Deserialize<List<string>>(json);
46             if (ids == null || ids.Count == 0) return;
47             foreach (var id in ids)
48                 _activeBuildings.Add(id);
49             EmergencyChanged?.Invoke(this, new EmergencyChangedEventArgs {
                BuildingId = null, IsActive = true });
50         }
51         catch { }
52     }
53
54     private void Save()
55     {
56         try
57         {
58             var json = JsonSerializer.Serialize(_activeBuildings.ToList());
59             File.WriteAllText(StatePath, json);
60         }
61         catch { }
62     }
63
64     public void Activate(string buildingId)
65     {
66         if (_activeBuildings.Add(buildingId))
67         {
68             Save();
69             EmergencyChanged?.Invoke(this, new EmergencyChangedEventArgs {
                BuildingId = buildingId, IsActive = true });
70         }

```

```

71     }
72
73     public void Deactivate(string buildingId)
74     {
75         if (_activeBuildings.Remove(buildingId))
76         {
77             Save();
78             EmergencyChanged?.Invoke(this, new EmergencyChangedEventArgs {
79                 BuildingId = buildingId, IsActive = false });
80         }
81     }
82
83     public List<NavNode> FindNearestExitRoute(NavNode start, NavGraph graph,
84         ISet<string>? excludeIds)
85     {
86         var exits = graph.Nodes
87             .Where(n => (n.IsExit || n.IsEvacuationExit)
88                 && n.BuildingId == start.BuildingId
89                 && excludeIds?.Contains(n.Id) != true)
90             .ToList();
91         if (!exits.Any())
92             exits = graph.Nodes
93                 .Where(n => (n.IsExit || n.IsEvacuationExit) && excludeIds?.
94                     Contains(n.Id) != true)
95                 .ToList();
96         if (!exits.Any()) return new();
97
98         var dist = new Dictionary<string, double>();
99         var prev = new Dictionary<string, string?>();
100         var queue = new SortedSet<(double d, string id)>(Comparer<(double d,
101             string id)>.Create(
102             (a, b) => a.d != b.d ? a.d.CompareTo(b.d) : string.Compare(a.id,
103                 b.id, StringComparison.Ordinal));
104
105         foreach (var n in graph.Nodes)
106         {
107             dist[n.Id] = double.MaxValue;
108             prev[n.Id] = null;
109         }
110         dist[start.Id] = 0;
111         queue.Add((0, start.Id));
112
113         var nodeMap = graph.Nodes.ToDictionary(n => n.Id);
114         var adj = BuildAdjacency(graph);
115
116         while (queue.Count > 0)
117         {
118             var (d, uid) = queue.Min;
119             queue.Remove(queue.Min);
120
121             if (!adj.TryGetValue(uid, out var neighbours)) continue;
122             foreach (var (nid, w) in neighbours)
123             {

```

```

119         if (nid != start.Id && excludeIds?.Contains(nid) == true)
120             continue;
121
122         var alt = d + w;
123         if (alt < dist[nid])
124         {
125             queue.Remove((dist[nid], nid));
126             dist[nid] = alt;
127             prev[nid] = uid;
128             queue.Add((alt, nid));
129         }
130     }
131
132     var bestExit = exits.OrderBy(e => dist.GetValueOrDefault(e.Id, double
133         .MaxValue)).FirstOrDefault();
134     if (bestExit == null || dist[bestExit.Id] == double.MaxValue) return
135         new();
136
137     var path = new List<NavNode>();
138     string? cur = bestExit.Id;
139     while (cur != null)
140     {
141         if (nodeMap.TryGetValue(cur, out var node)) path.Add(node);
142         prev.TryGetValue(cur, out cur);
143     }
144     path.Reverse();
145     return path;
146 }
147
148 private static Dictionary<string, List<(string id, double w)>>
149     BuildAdjacency(NavGraph graph)
150 {
151     var adj = new Dictionary<string, List<(string, double)>>();
152     foreach (var n in graph.Nodes)
153         adj[n.Id] = new();
154
155     var nodeMap = graph.Nodes.ToDictionary(n => n.Id);
156     foreach (var e in graph.Edges)
157     {
158         if (!nodeMap.TryGetValue(e.FromId, out var a)) continue;
159         if (!nodeMap.TryGetValue(e.ToId, out var b)) continue;
160         double w = Math.Sqrt(Math.Pow(a.X - b.X, 2) + Math.Pow(a.Y - b.Y,
161             2));
162         adj[e.FromId].Add((e.ToId, w));
163         adj[e.ToId].Add((e.FromId, w));
164     }
165     return adj;
166 }
167 }

```

Автор ВКР

(подпись, дата)

В. А. Старков

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска